CLOUDSTORE: PRODUCTION-READY HIGHLY AVAILABLE AWS MULTI-TIER ARCHITECTURE

A Dissertation Submitted To

BIGLEARN TRAINING INSTITUTE

TIRUCHIRAPPALLI



In Partial Fulfillment Of The Requirement For

The Award Of The Degree Of

CLOUD COMPUTING

GUIDED BY

NANCY JOSHLIN.A

BY

VISHAL B

CLOUDSTORE

AWS MULTI-TIER CLOUD INFRASTRUCTURE

PROJECT IMPLEMENTATION & EVIDENCE REPORT

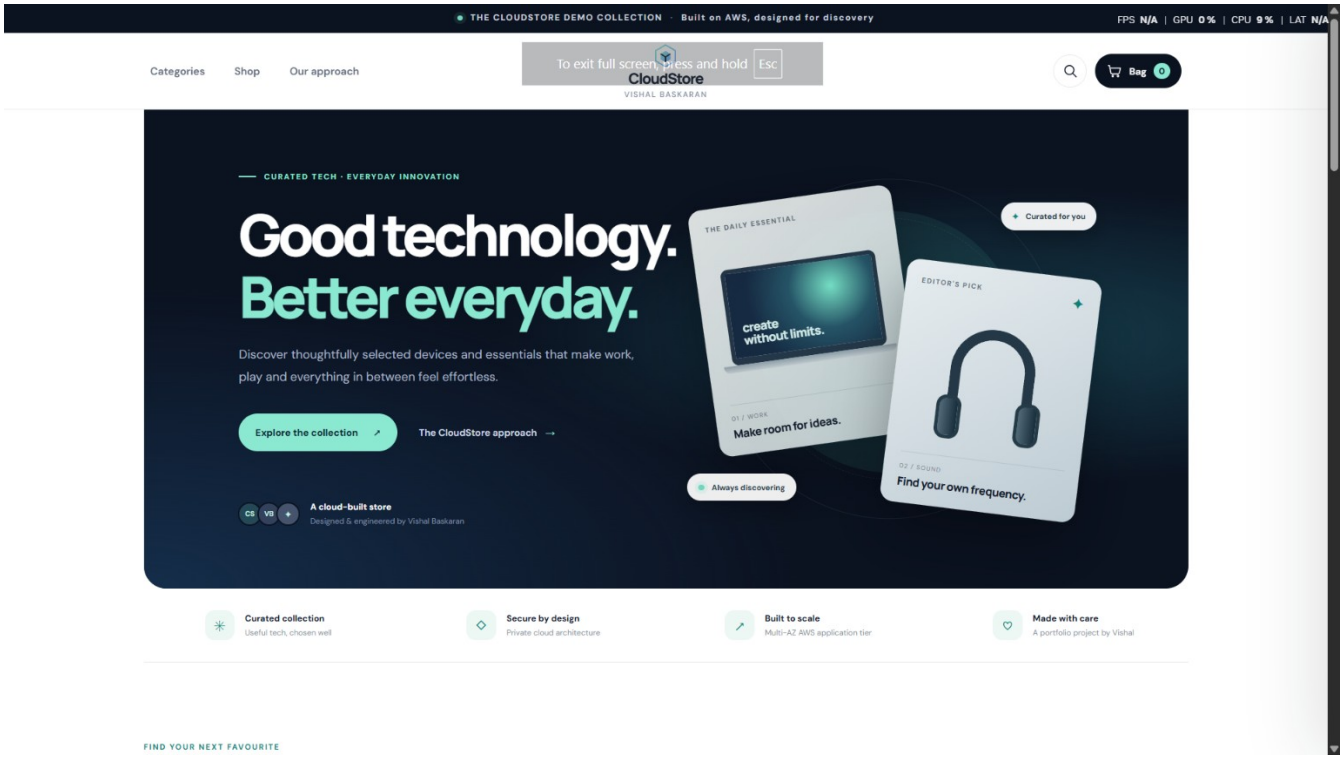


Figure 1. CloudStore storefront - deployed application interface

PROJECT PROFILE

A Flask e-commerce demonstration deployed on AWS. The solution uses an internet-facing Application Load Balancer, EC2 Auto Scaling application tier, Nginx and Gunicorn, and Amazon RDS for MySQL.

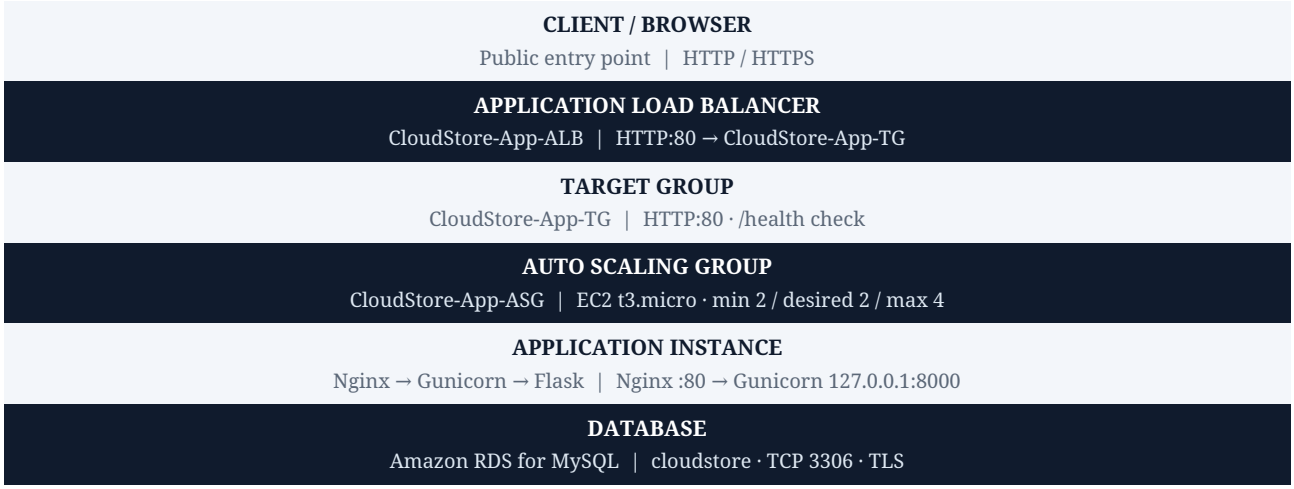
Project item	Observed / configured value	Project purpose
Cloud / region	AWS ap-southeast-2 (Sydney)	Host the CloudStore application stack.
Web application	Flask + Gunicorn + Nginx	Serve the storefront and JSON API.
Compute and data	EC2 Auto Scaling + RDS MySQL	Provide application capacity and persistent catalogue data.

Project owner: Vishal Baskaran | Report date: 02 November 2026

01 / PROJECT OVERVIEW

Executive Summary & Solution Architecture

CloudStore is a demonstration e-commerce application built with a Flask backend and a browser-based storefront. The AWS design separates public traffic entry, application processing, and relational data storage. An Application Load Balancer forwards requests to a target group associated with an EC2 Auto Scaling Group. Nginx acts as the web reverse proxy, Gunicorn runs the Flask application, and Amazon RDS for MySQL stores product and demo order data.



Traffic flows from the browser through the ALB and target group to an application instance. Nginx forwards the request to Gunicorn on localhost port 8000, where Flask handles the route. The application tier initiates database connections; browser clients do not connect directly to MySQL.

Design objective	Implementation evidence	Why it matters
Availability	Two Availability Zones in the ASG network configuration.	Maintains an application fleet across more than one zone.
Elasticity	ASG min 2, desired 2 and max 4; target tracking policy shown at 50% average CPU.	Allows capacity to change within defined limits.
Isolation	Separate ALB, application and database security groups.	Restricts traffic to the intended tier-to-tier paths.
Validation	ALB health/API requests returned HTTP 200 in the supplied test output.	Confirms the observed endpoint and API paths responded successfully.

SCOPE NOTE

This is a portfolio and learning project report. It records the configuration and test output visible in the supplied evidence. It does not claim a formal production security certification or real payment processing.

02 / FRONTEND EXPERIENCE

Storefront Landing Page

The landing page introduces the CloudStore brand and provides the primary navigation path into the catalogue. The first viewport combines the main value statement, a product-discovery call to action, and secondary navigation.

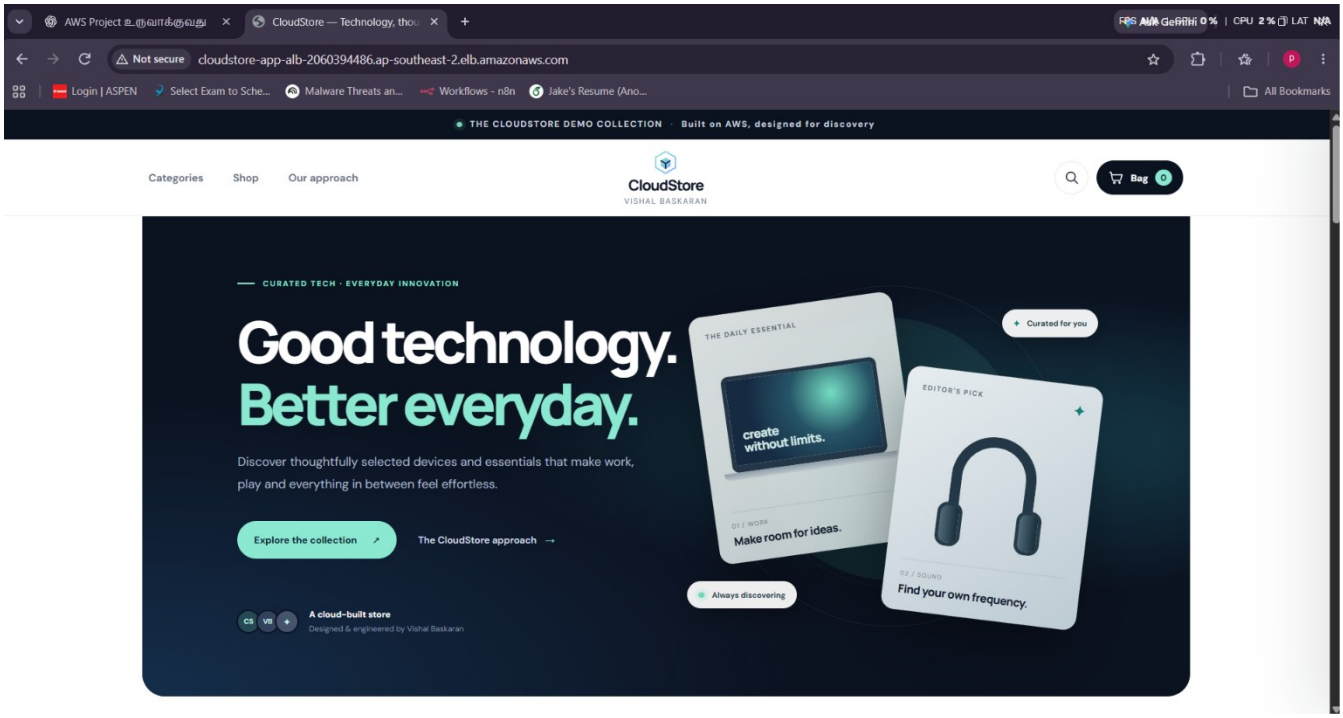


Figure 1. Storefront Landing Page - Library screenshot proof

The header links users to Categories, Shop and Our approach. The hero gives the site a clear starting point and offers a route into the product collection. The screenshot records the deployed browser view during the project test session.

Area	Observed UI element	Purpose
Header	Categories / Shop / Our approach	Primary navigation between store sections.
Hero	Main headline and collection CTA	Directs visitors towards catalogue discovery.
Brand block	CloudStore identity and project credit	Establishes the demo-store identity.
Feature strip	Curated collection / secure by design / built to scale	Summarises product and cloud positioning.

The home screen is presentation-focused. It is the browser entry point, not a separate service tier; application delivery still depends on the ALB and application fleet described in the architecture section.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

The landing page is the presentation layer for the application. Static CSS and JavaScript assets were also requested through the ALB and returned HTTP 200, so the page render depends on both the HTML response and its supporting assets. The hero CTA and top navigation are the visible user routes into the rest of the demo experience.

Component	Observed value	Purpose
HTML document	GET /	Returns the page template and initial storefront shell.
Stylesheet	/static/css/styles.css	Applies typography, spacing, cards and responsive styling.
Browser script	/static/js/app.js	Supports catalogue interaction and cart UI.
Public route	ALB DNS name	Provides a single public entry point for the browser.

02 / FRONTEND EXPERIENCE

Category Discovery

The category view groups the product catalogue into clear shopping areas. It offers a faster route to relevant products than scanning every product card.

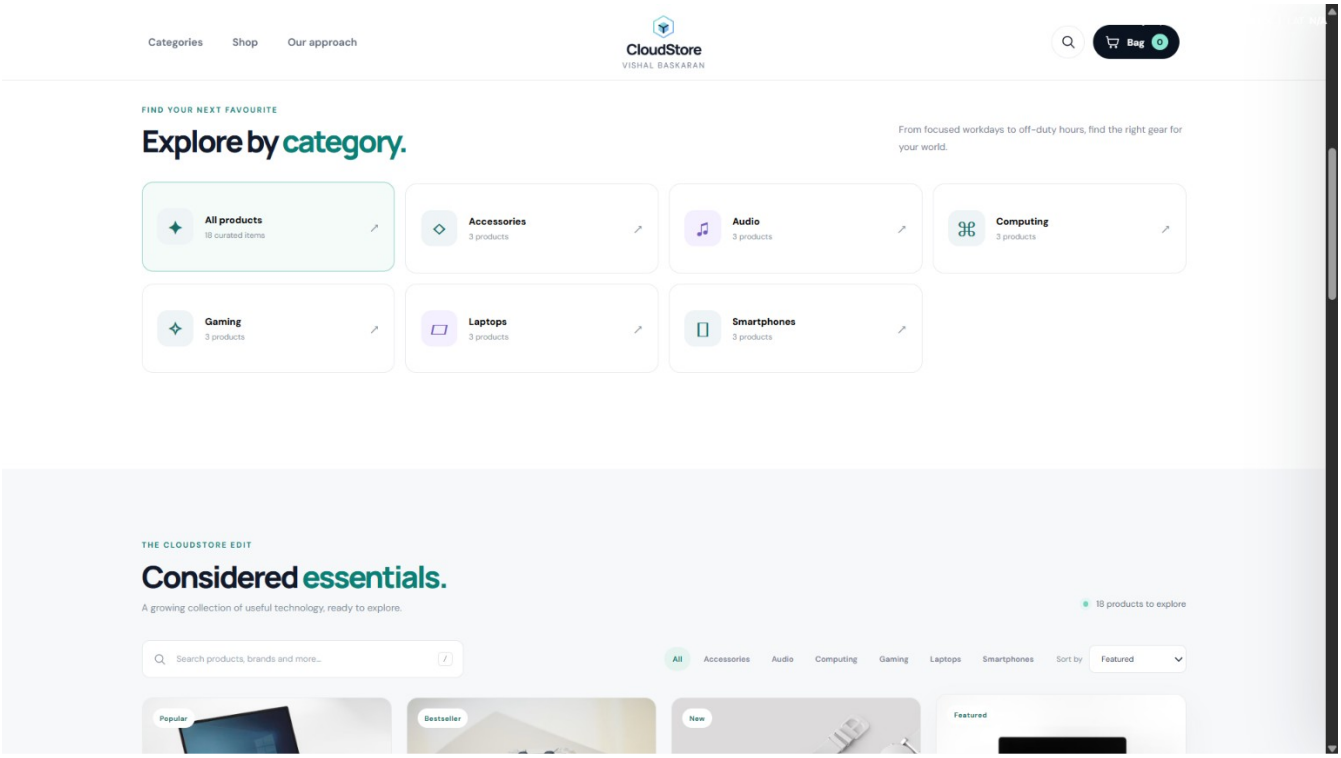


Figure 2. Category Discovery - Library screenshot proof

The category cards include brief descriptions and product counts. The category experience is supported by the backend /api/categories route, which returned six category records with HTTP 200 during the supplied API tests.

Category	Product count	Browsing role
Accessories	3	Chargers, wearables and accessories.
Audio	3	Headphones and audio devices.
Computing	3	Monitors and peripherals.
Gaming / Laptops / Smartphones	3 each	Completes the six-category, 18-product demo catalogue.

Category navigation is a frontend action backed by a JSON API response. The API evidence confirms six category records, each associated with three sample products in the captured output.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

The category screen works as a discovery layer. A visitor can choose a broad category first, then review the matching product cards in the catalogue. The six category names visible in the API output are Accessories, Audio, Computing, Gaming, Laptops and Smartphones. Each category contained three sample products in the captured response.

Review focus	Expected behaviour	Evidence meaning
Category selection	User can select a category card or filter	User confirmed category controls worked during manual frontend testing.
Category metadata	Name, description and product count	Visible category cards provide context before product browsing.
API response	Six JSON category records	Backend category retrieval returned HTTP 200.

02 / FRONTEND EXPERIENCE

Curated Product Collection

This screen shows product cards arranged as a curated collection. Cards combine product imagery, names, short descriptions and prices, helping visitors compare products visually.

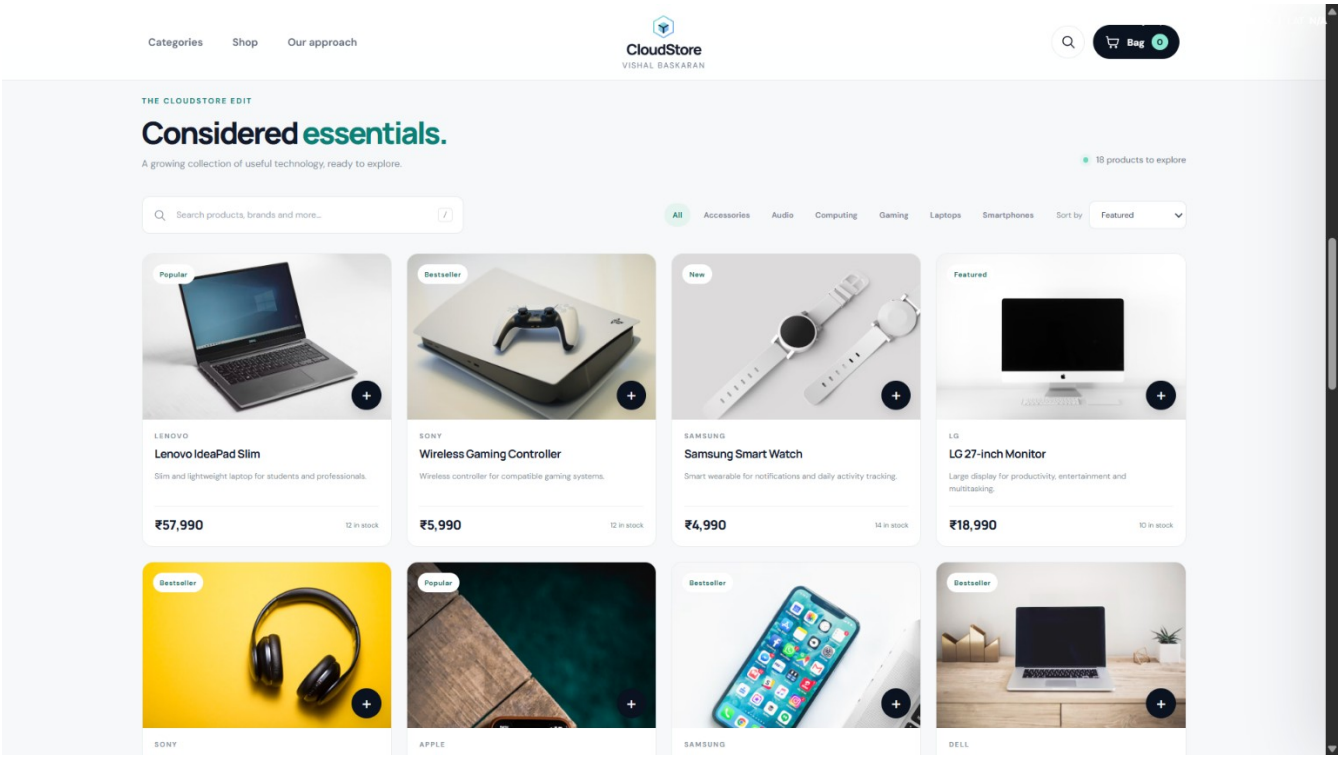


Figure 3. Curated Product Collection - Library screenshot proof

The repeated card layout gives the catalogue a consistent rhythm. The visible price and stock labels are sample catalogue data returned by the API and displayed by the browser.

Card element	Information shown	Why it matters
Product image	Visual product identity	Lets users scan items quickly.
Name and price	Product title and displayed price	Supports basic product comparison.
Description	Short product summary	Adds context to each listing.
Badge	Popular / Bestseller / New / Featured	Highlights selected catalogue items.

The product cards are data-driven. When catalogue values change in the database, the API response is the authoritative source consumed by the storefront; static screenshot values should be treated as captured demo data.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

Product cards are a reusable presentation component. The same visual structure is applied to products from different groups so the user can compare names, brands, short descriptions and prices without learning a new interaction pattern for each category. The API payload includes a product identifier, category, badge, stock quantity and image URL in addition to the customer-facing text.

Product-card field	Source meaning	UI use
product_name / brand	Catalogue product attributes	Primary title and brand label.
price / stock_quantity	Sample commercial and stock values	Price display and stock note.
badge / featured	Merchandising metadata	Optional product label and curated ordering.

02 / FRONTEND EXPERIENCE

Shop Catalogue View

The shop view displays a broader set of product cards and provides browsing controls for the demo catalogue. The screenshot documents the product grid as it appeared during frontend validation.

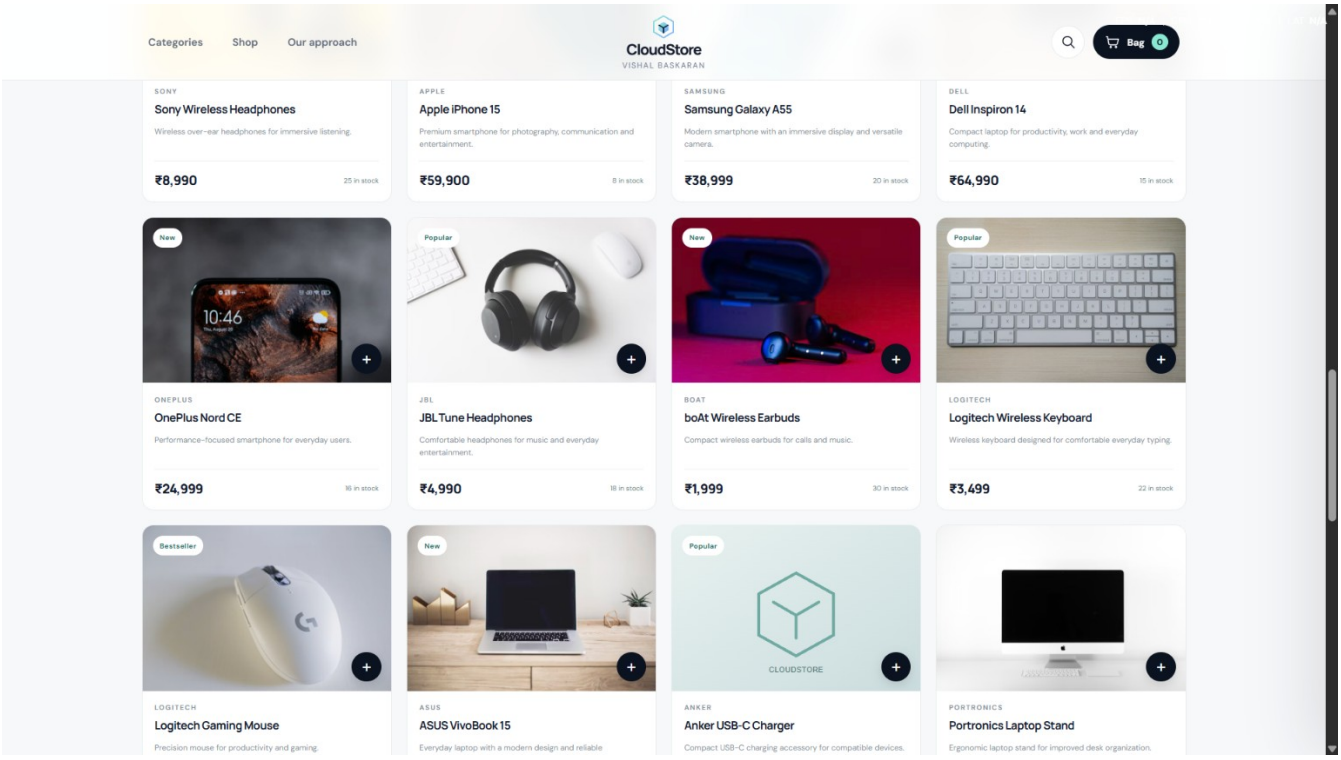


Figure 4. Shop Catalogue View - Library screenshot proof

The visible controls include search, category filters and a sort selection. During manual testing, the user confirmed that catalogue browsing, search and category filtering worked as expected.

Control / element	Role in page	Test context
Search field	Find items by product text	Search interaction was manually tested.
Category controls	Narrow the visible product set	Category route returned HTTP 200.
Sort selection	Organise visible cards	Demonstrates catalogue control.
Product grid	Displays catalogue records	18 sample products were available.

Manual interaction checks confirmed search and filter operation. The report distinguishes those browser checks from the server-side API tests, which are recorded separately in the validation pages.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

Catalogue controls support a basic browse workflow: narrow by category, locate an item by text, then change the sort selection. The browser screenshot shows these controls as part of one cohesive page. The API test output separately demonstrates that the backend can supply the complete product collection through /api/products.

Control	User action	Review question
Search	Enter a product or brand term	Does the visible product set narrow to a matching item?
Category filter	Select one category	Are only matching products shown?
Sort selection	Choose a sort option	Does the order of product cards change consistently?

02 / FRONTEND EXPERIENCE

Product Listing Layout

This screenshot captures another product-grid arrangement and demonstrates that the same CloudStore visual system is applied across multiple product groups.

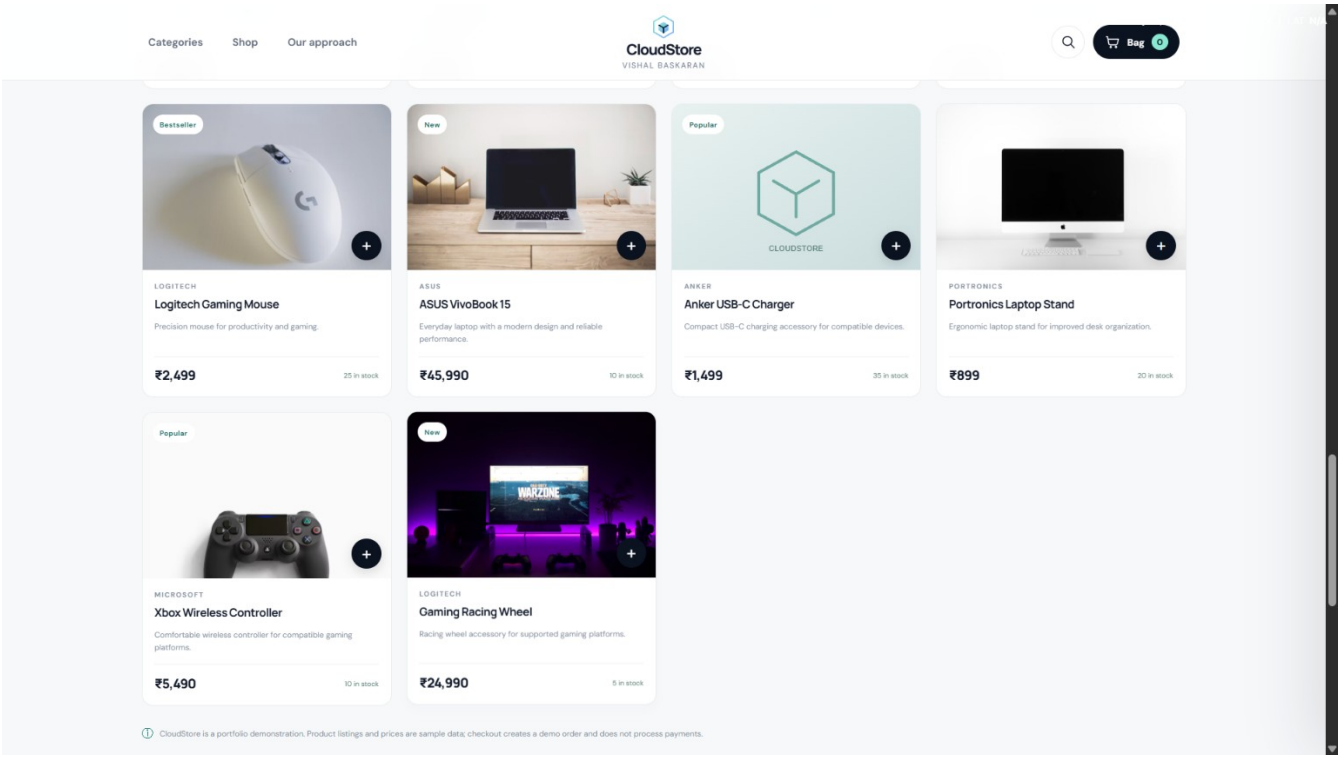


Figure 5. Product Listing Layout - Library screenshot proof

Product images, brand labels, descriptions, badge values and prices are displayed together. The website uses the product API for data and the browser for layout and interaction.

Displayed data	Origin	User-facing result
Name, brand and price	Product API records	Consistent product-card information.
Category and badge	Product/category fields	Provides context and merchandising cues.
Image and description	Catalogue fields	Makes the cards visually descriptive.
Stock availability	stock_quantity field	Shows demo inventory counts.

The listing page demonstrates data presentation rather than stock reservation. Product stock values are displayed sample counts; this project evidence does not establish a production-grade inventory locking workflow.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

The listing uses a stable hierarchy: product image first, then brand and name, then a concise description, then price and stock information. Badges such as Popular, Bestseller, New and Featured provide visual merchandising, but they are catalogue labels rather than independent product ratings or endorsements. The screenshot records the visual presentation at the time of testing.

Data field	Why it is displayed	Validation boundary
Image URL	Visual recognition	Image hosting is external to the AWS application tier.
Brand and product name	Identify the catalogue item	Text is returned by the products API.
Price and stock	Show sample demo values	Values are demo catalogue data, not live merchant inventory.

02 / FRONTEND EXPERIENCE

Brand and Design Story

The approach section communicates the thinking behind the storefront and extends the page beyond a product grid. It positions the application as a coherent portfolio product.

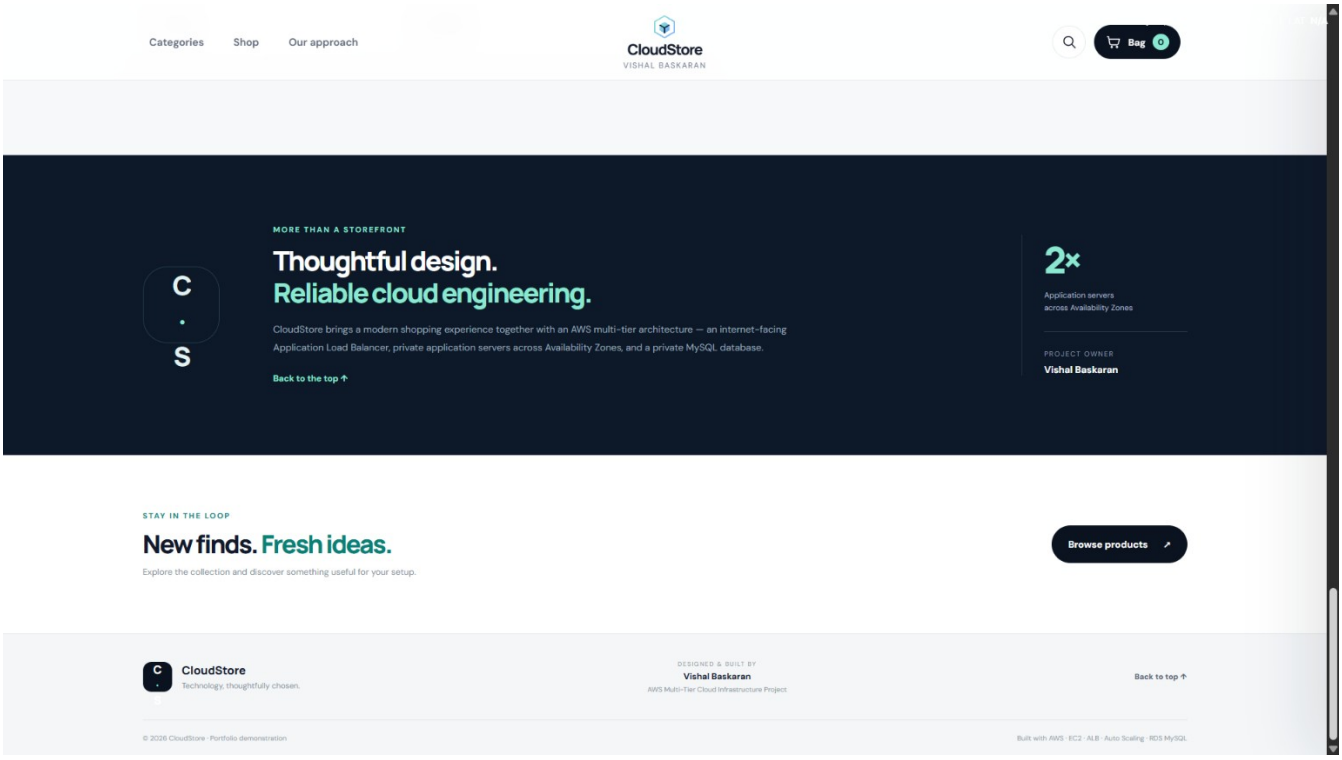


Figure 6. Brand and Design Story - Library screenshot proof

The design uses a dark feature panel, concise messaging and teal accent typography. The same colour and type choices recur in the header, product collection and supporting content.

Design choice	Observed treatment	Contribution
Dark background	High-contrast feature panel	Separates the story section from product grids.
Short messaging	Concise brand statements	Explains the design approach.
Consistent accents	Teal and repeated typography	Maintains a coherent visual identity.
Project attribution	Owner credit in the page	Identifies the project as a portfolio build.

The approach section communicates the non-functional goals that shaped the build: a coherent interface, a secured multi-tier network path, and compute capacity managed by Auto Scaling.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

The approach section helps explain why the storefront and cloud architecture are presented together. The website is deliberately a demo storefront, while the supporting message highlights the private application tier and AWS-managed scaling concept. This is useful in a portfolio walkthrough because it connects a visible browser experience to the infrastructure behind it.

Design element	Observed choice	Presentation effect
Feature panel	Dark background with high-contrast heading	Separates the story section from the catalogue.
Accent colour	Teal / mint highlighting	Repeats the visual identity across sections.
Architecture statement	Multi-AZ application tier	Connects brand story with the infrastructure design.

02 / FRONTEND EXPERIENCE

Editorial / Supporting Section

The supporting content area provides a secondary information section beneath the main storefront experience. It shows that the site includes brand and supporting content in addition to product browsing.

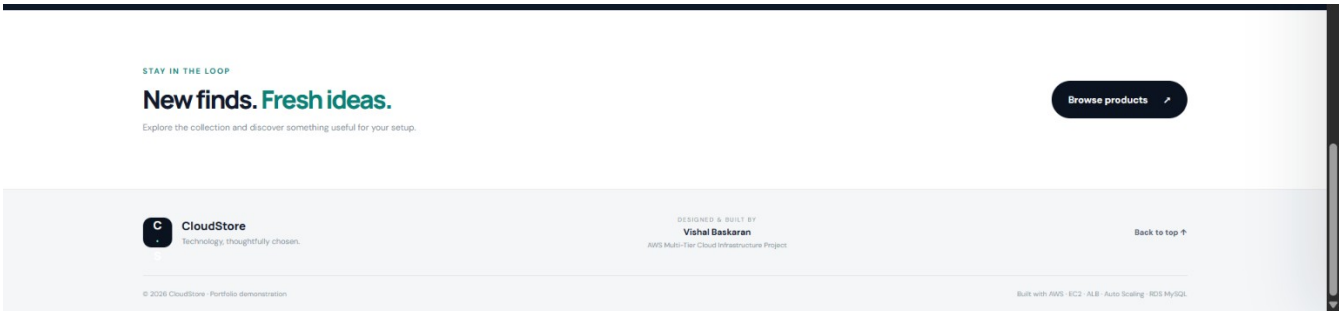


Figure 7. Editorial / Supporting Section - Library screenshot proof

The section includes a short discovery message, a browse-products action and the site footer. These details improve page completeness and provide a visual endpoint to the storefront.

Content region	Observed purpose	Project relevance
Editorial heading	Introduces supporting content	Adds depth beyond shopping.
Browse action	Returns visitors to products	Creates a useful navigation route.
Footer identity	CloudStore brand and owner credit	Completes the page layout.
Portfolio note	AWS project attribution	Communicates the demonstration context.

This lower-page area is a supporting experience, not a separate backend microservice. It relies on the same deployed web application and static frontend assets.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

The supporting section is the lower-page completion of the storefront experience. It contains a short editorial message, a browse-products action and the footer with CloudStore identity, project attribution and a back-to-top link. It does not represent a separate backend service; it is part of the same Flask-rendered application and its static frontend assets.

Footer element	Purpose	Portfolio relevance
Brand identity	CloudStore mark and short tagline	Keeps the store identifiable at the bottom of the page.
Project credit	Designed and built by project owner	Makes authorship explicit.
Portfolio context	AWS multi-tier project note	Connects the storefront to the cloud implementation.

The footer also serves as a navigation and attribution boundary. The browse action returns visitors to the catalogue, while the back-to-top link provides a convenient route to the primary navigation without changing the application route or contacting a separate service.

Footer / navigation check	Observed behaviour	Why it matters
Browse products	Provides a route back to the catalogue.	Keeps the shopping journey continuous.
Back to top	Returns the visitor to the top section.	Improves movement across a long storefront page.
Brand and owner credit	Shows CloudStore identity and project authorship.	Makes the portfolio demonstration identifiable.
Copyright / demo note	Identifies the site as a portfolio demonstration.	Sets appropriate expectations for sample data.

02 / FRONTEND EXPERIENCE

Shopping Bag / Cart

The cart screenshot captures a selected product and the shopping-bag panel. It documents the visible state before the checkout form is submitted.

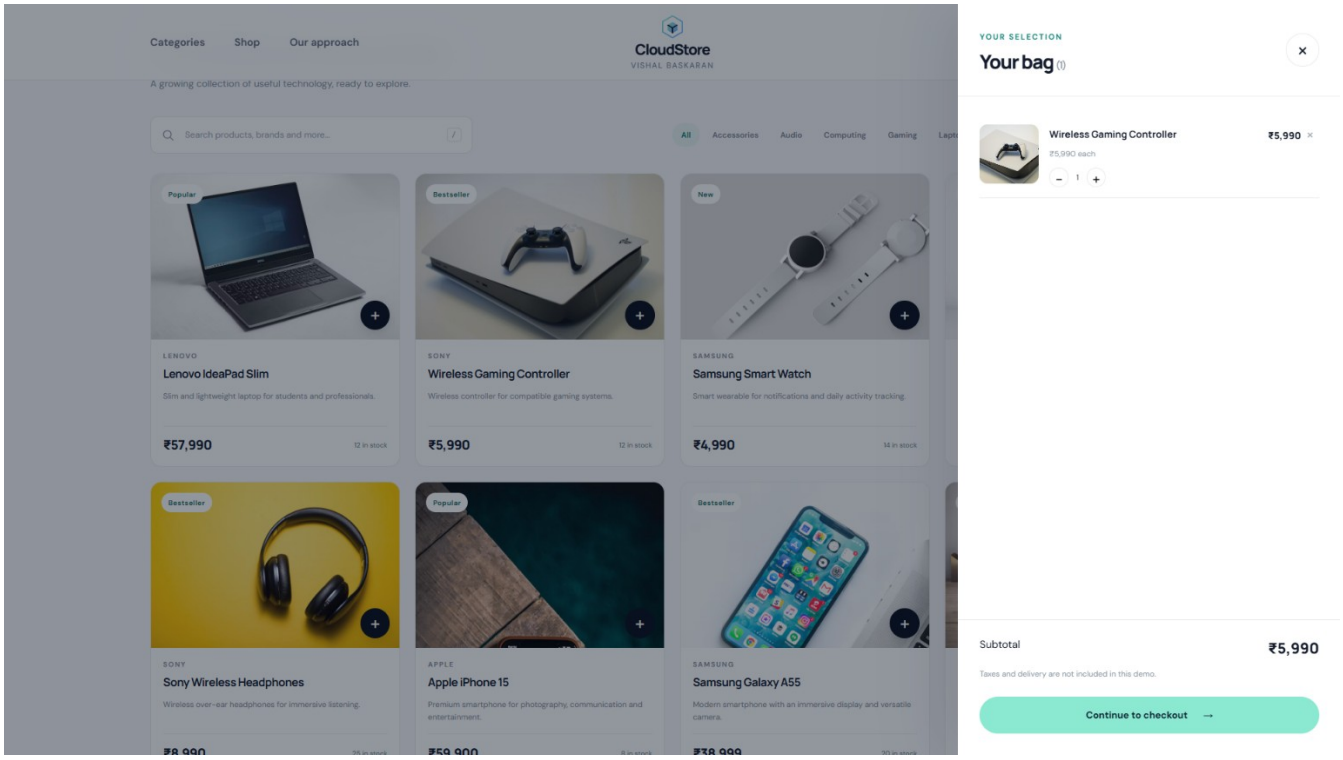


Figure 8. Shopping Bag / Cart - Library screenshot proof

Manual frontend testing confirmed the cart display, cart count, quantity controls and remove action. Checkout remains a demo flow and explicitly states that no payment is collected. The final order-submit action was not confirmed successful, so this report does not claim a completed order transaction.

Cart feature	Expected behaviour	Test note
Bag contents	Show selected product and price	User confirmed the cart display worked.
Quantity / removal	Update or remove selected items	User confirmed cart controls worked.
Demo checkout	Collect customer form details	No payment is collected by this demo.
Order submit	Send order details to backend	A temporary client-side reset error was observed; no successful order is claimed.

The cart is suitable for demonstration and interaction testing. Because the final submit step showed a JavaScript reset error, use the project as a storefront and infrastructure demo rather than a completed transactional commerce platform.

FRONTEND REVIEW

Implementation Notes & Evidence Interpretation

The bag panel shows the item selected by the visitor and provides quantity and removal controls. The checkout dialog explains that it records a demo order in the CloudStore MySQL database and that no payment is collected. This is a demonstration path only. During the supplied test, the user confirmed the cart controls but the final submit step displayed a client-side reset error; the report therefore does not claim that a database order was successfully created.

Checkout field	Purpose	Data-handling consideration
Full name / email	Demo customer identification	Use test data only in public screenshots.
Phone / address	Optional contact and delivery details	Avoid exposing personal values in published evidence.
Order total	Displays cart total for demo review	Not a card charge or payment confirmation.

03 / DATA TIER

Database Structure & Entity Definitions

The CloudStore database is hosted on Amazon RDS for MySQL. The application uses the database name cloudstore and connects over TCP port 3306. The runtime account is a dedicated application account, separate from the RDS master account. The supplied terminal screenshot records the database table definitions used during the project build.

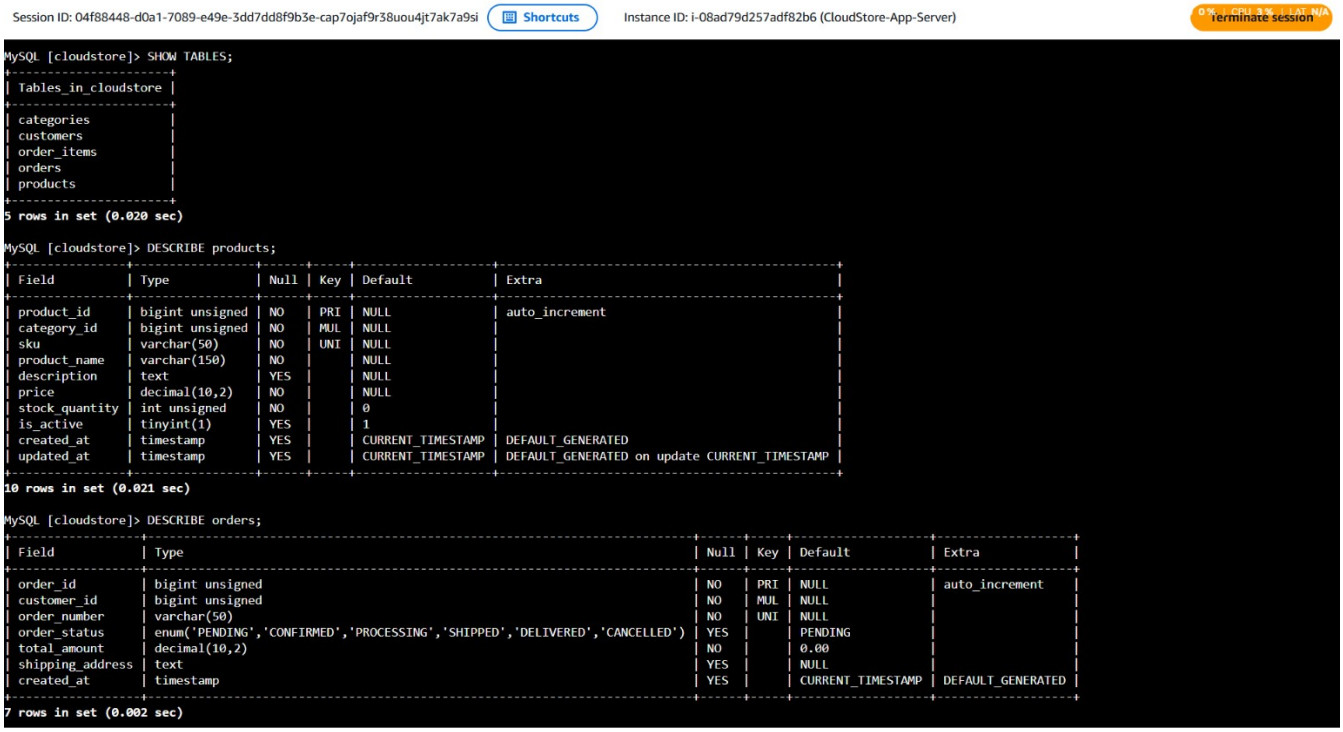


Figure 9. MySQL table definitions and column metadata

The visible schema evidence includes the product and supporting e-commerce entities. The live database remains the authority for field types and constraints; a schema migration should always be checked against the deployed database before it is run.

Database concern	Design / evidence focus
Catalogue	Products and categories form the core browsing data.
Orders	Orders and order_items support the demo checkout data model.
Customers	Customer contact fields are stored separately from catalogue records.
Integrity	Use the live schema as the source of truth for field types and constraints.

DATA-LAYER BOUNDARY

The RDS security group permits MySQL ingress from the application security group, not directly from arbitrary public internet addresses.

03 / DATA TIER

Product Records & Catalogue Query

The database console output shows product records joined with category information. The result includes product identifiers, SKUs, product names, category labels, prices, stock quantities and merchandising badges. This supports the catalogue displayed in the browser and links stored rows to the API product response.

```
MySQL [cloudstore]> SELECT
    ->     c.category_name,
    ->     p.product_id,
    ->     p.sku,
    ->     p.brand,
    ->     p.product_name,
    ->     p.price,
    ->     p.stock_quantity,
    ->     p.badge
    -> FROM products p
    -> JOIN categories c
    ->     ON p.category_id = c.category_id
    -> ORDER BY c.category_id, p.product_id;
+---------------+------------+------------+------------+---------------------------+----------+----------------+------------+
| category_name | product_id | sku        | brand      | product_name              | price    | stock_quantity | badge      |
+---------------+------------+------------+------------+---------------------------+----------+----------------+------------+
| Laptops       |          1 | CS-LAP-001 | Dell       | Dell Inspiron 14          | 64990.00 |             15 | Bestseller |
| Laptops       |          2 | CS-LAP-002 | Lenovo     | Lenovo IdeaPad Slim       | 57990.00 |             12 | Popular    |
| Laptops       |          3 | CS-LAP-003 | ASUS       | ASUS VivoBook 15          | 45990.00 |             10 | New        |
| Smartphones   |          4 | CS-PHN-001 | Samsung    | Samsung Galaxy A55        | 38999.00 |             20 | Bestseller |
| Smartphones   |          5 | CS-PHN-002 | Apple      | Apple iPhone 15           | 59900.00 |              8 | Popular    |
| Smartphones   |          6 | CS-PHN-003 | OnePlus    | OnePlus Nord CE           | 24999.00 |             16 | New        |
| Audio         |          7 | CS-AUD-001 | Sony       | Sony Wireless Headphones  |  8990.00 |             25 | Bestseller |
| Audio         |          8 | CS-AUD-002 | JBL        | JBL Tune Headphones       |  4990.00 |             18 | Popular    |
| Audio         |          9 | CS-AUD-003 | boAt       | boAt Wireless Earbuds     |  1999.00 |             30 | New        |
| Computing     |         10 | CS-CMP-001 | LG         | LG 27-inch Monitor        | 18990.00 |             10 | Featured   |
| Computing     |         11 | CS-CMP-002 | Logitech   | Logitech Wireless Keyboard|  3499.00 |             22 | Popular    |
| Computing     |         12 | CS-CMP-003 | Logitech   | Logitech Gaming Mouse     |  2499.00 |             25 | Bestseller |
| Accessories   |         13 | CS-ACC-001 | Samsung    | Samsung Smart Watch       |  4990.00 |             14 | New        |
| Accessories   |         14 | CS-ACC-002 | Anker      | Anker USB-C Charger       |  1499.00 |             35 | Popular    |
| Accessories   |         15 | CS-ACC-003 | Portronics | Portronics Laptop Stand   |   899.00 |             20 | NULL       |
| Gaming        |         16 | CS-GAM-001 | Sony       | Wireless Gaming Controller|  5990.00 |             12 | Bestseller |
| Gaming        |         17 | CS-GAM-002 | Microsoft  | Xbox Wireless Controller  |  5490.00 |             10 | Popular    |
| Gaming        |         18 | CS-GAM-003 | Logitech   | Gaming Racing Wheel       | 24990.00 |              5 | New        |
+---------------+------------+------------+------------+---------------------------+----------+----------------+------------+
18 rows in set (0.004 sec)
```

Figure 10. Product data query showing catalogue rows and category labels

The screenshot demonstrates populated sample data for the demo store. The catalogue contains six product categories and 18 products according to the API output captured during application testing. This query is database evidence, not a live inventory report.

Field	Meaning in the catalogue
product_id / SKU	Internal product key and store-keeping code.
product_name / category	Customer-visible product and catalogue group.
price / stock_quantity	Demo price and sample inventory count.
badge / brand	Additional merchandising and product metadata.

The API returned products in JSON form. The web layer can therefore reuse the same catalogue data for the grid, search and product-detail display without embedding database queries in the browser.

03 / DATA TIER

Category Aggregation & Data Validation

A category aggregation query was used to verify that products are associated with the expected catalogue groups. The visible output shows category names alongside product totals. This is a compact consistency check between the products and categories tables.

```
MySQL [cloudstore]> SELECT COUNT(*) AS total_products
    -> FROM products;
+----------------+
| total_products |
+----------------+
|             18 |
+----------------+
1 row in set (0.001 sec)

MySQL [cloudstore]> SELECT
    ->     c.category_name,
    ->     COUNT(p.product_id) AS total_products
    -> FROM categories c
    -> LEFT JOIN products p
    ->     ON c.category_id = p.category_id
    -> GROUP BY c.category_id, c.category_name
    -> ORDER BY c.category_id;
+---------------+----------------+
| category_name | total_products |
+---------------+----------------+
| Laptops       |              3 |
| Smartphones   |              3 |
| Audio         |              3 |
| Computing     |              3 |
| Accessories   |              3 |
| Gaming        |              3 |
+---------------+----------------+
6 rows in set (0.002 sec)

MySQL [cloudstore]>
```

Figure 11. SQL category aggregation and product-count output

During API validation, /api/categories returned six category records and /api/products returned 18 product records. Both endpoints returned HTTP 200 through the Application Load Balancer. Together, the SQL screenshot and API responses provide evidence across the data and application layers.

Validation item	Observed result	Interpretation
/api/categories	HTTP 200; six records	Category retrieval succeeded.
/api/products	HTTP 200; 18 records	Product catalogue retrieval succeeded.
/api/products/1	HTTP 200; Dell Inspiron 14 record	Individual product lookup succeeded.
/api/health	application=ok; database=ok	Application and database checks reported healthy.

04 / COMPUTE & RUNTIME

Application Server Provisioning

The terminal screenshot documents application-server preparation, including package activity and the start of the deployment workflow. The application hosts use Amazon Linux 2023 and t3.micro in the demonstrated Auto Scaling configuration.

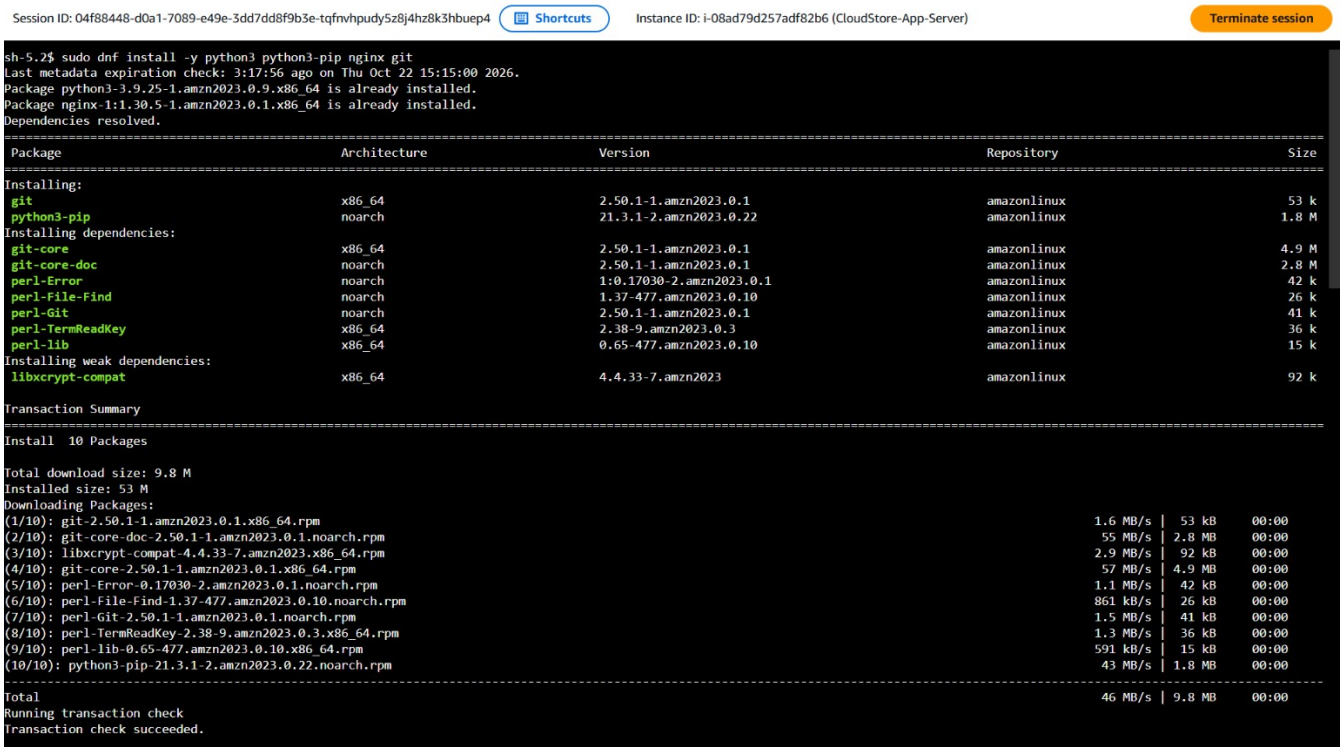


Figure. Application Server Provisioning - application host terminal evidence

The deployment process creates a dedicated Linux account named cloudstore and places the application under /opt/cloudstore. The application service is configured to run as this non-root account.

Item	Configured value	Purpose
Linux service user	cloudstore	Runs the app using a dedicated identity.
Application directory	/opt/cloudstore	Holds repository files and the Python environment.
Operating system	Amazon Linux 2023	Base platform used by the app tier.
Instance type	t3.micro	Small general-purpose instance for the demo.

Operational note: Auto Scaling instances should be treated as replaceable compute. All configuration required for a new instance must be reproducible from the launch template, repository, IAM instance profile and Secrets Manager secret.

04 / COMPUTE & RUNTIME

System Packages & Runtime Dependencies

This terminal evidence records package installation and dependency setup on the application host. The required software includes Git, Python 3, pip and Nginx.

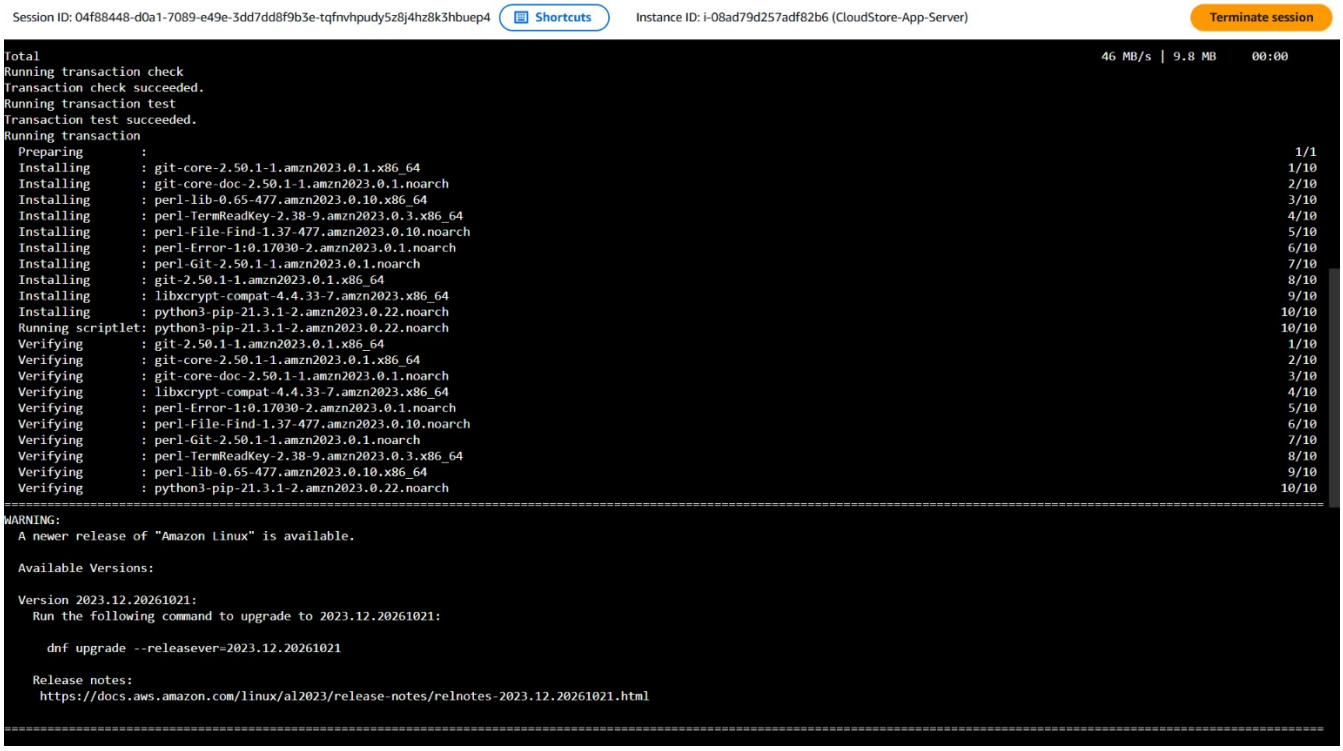


Figure. System Packages & Runtime Dependencies - application host terminal evidence

The Python virtual environment isolates application libraries from system Python. The application requirements include Flask, Gunicorn and mysql-connector-python.

Package	Role	Deployment use
Git	Source retrieval	Clones the CloudStore repository.
Python 3 / pip	Application runtime	Creates a virtual environment and installs requirements.
Nginx	HTTP reverse proxy	Receives traffic on port 80.
Gunicorn	WSGI server	Runs Flask workers behind Nginx.

Operational note: Auto Scaling instances should be treated as replaceable compute. All configuration required for a new instance must be reproducible from the launch template, repository, IAM instance profile and Secrets Manager secret.

04 / COMPUTE & RUNTIME

Application Repository & Python Environment

The application repository is cloned to the configured application directory and ownership is assigned to the dedicated service user. The setup then creates a virtual environment and installs the declared Python requirements.

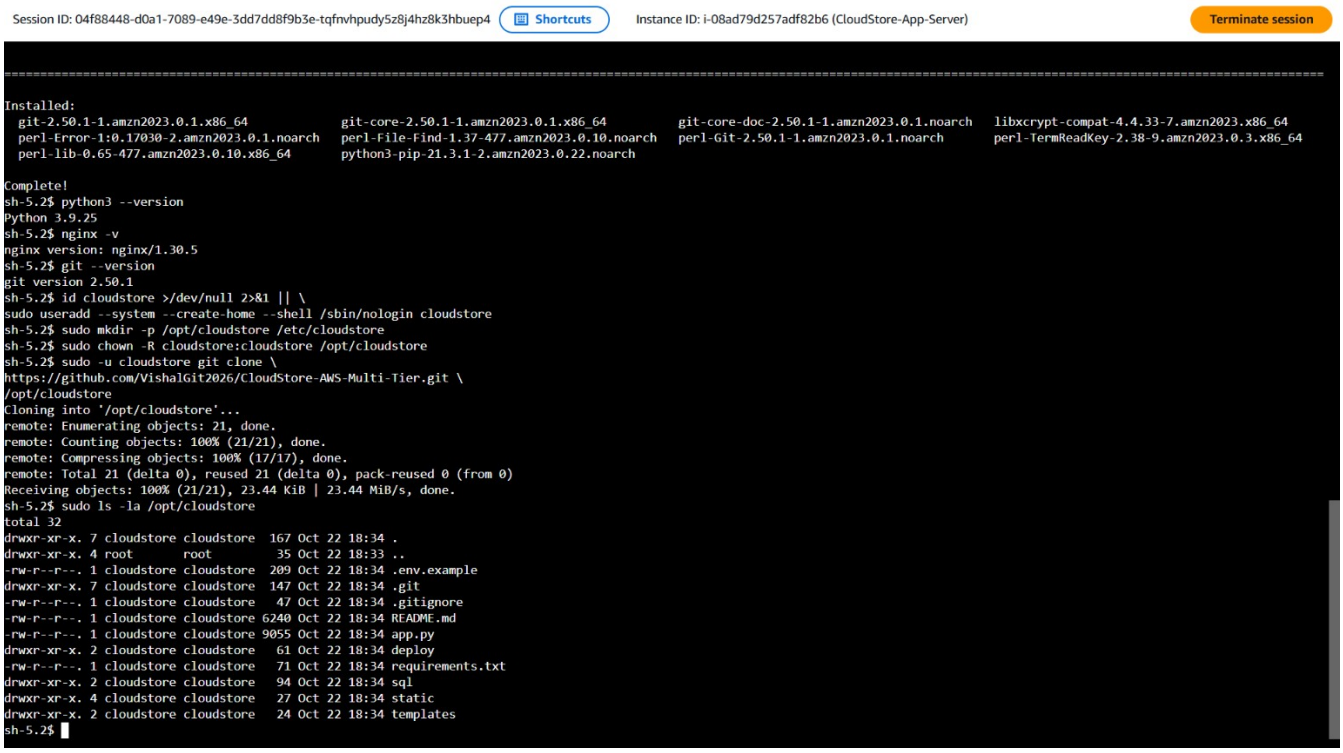


Figure. Application Repository & Python Environment - application host terminal evidence

The deployed project uses a Gunicorn systemd service to run the Flask application. The service binds to 127.0.0.1:8000, keeping the application worker behind the local Nginx reverse proxy.

Component	Configuration	Operational role
Virtual environment	/opt/cloudstore/venv	Isolates Python libraries.
Flask	Application framework	Implements CloudStore routes.
Gunicorn	app:app	Runs workers and threads on localhost.
Repository	CloudStore-AWS-Multi-Tier	Source for the deployed demo app.

Operational note: Auto Scaling instances should be treated as replaceable compute. All configuration required for a new instance must be reproducible from the launch template, repository, IAM instance profile and Secrets Manager secret.

04 / COMPUTE & RUNTIME

Service Configuration & Nginx Proxy

This screenshot records runtime configuration and installation output. The deployment configures a systemd unit for Gunicorn and installs the CloudStore Nginx configuration under /etc/nginx/conf.d.

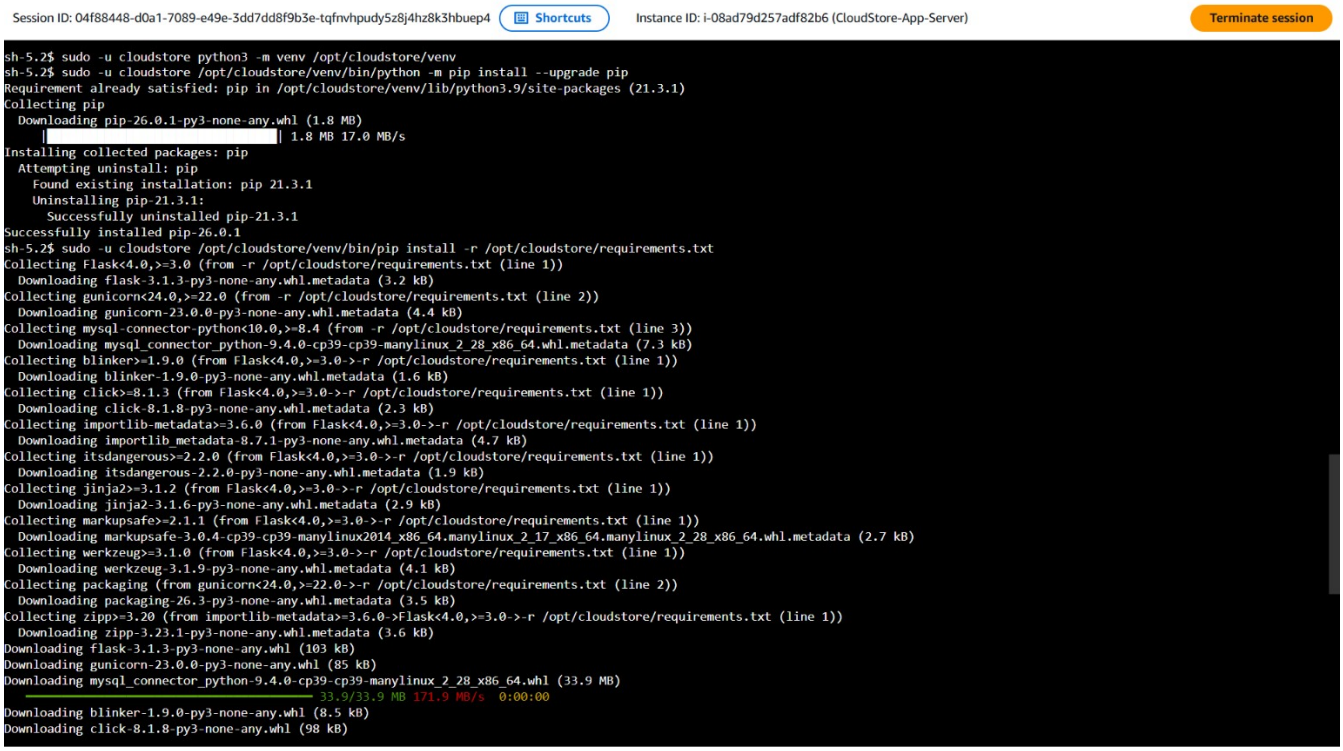


Figure. Service Configuration & Nginx Proxy - application host terminal evidence

Nginx receives HTTP requests on port 80 and proxies them to Gunicorn on localhost port 8000. The application health route is exposed at /health for the target-group health check.

Component	Port / path	Purpose
Nginx	TCP 80	Receives ALB requests and proxies to Gunicorn.
Gunicorn	127.0.0.1:8000	Executes the Flask application locally.
Health route	/health	Provides the target-group health-check response.
API health	/api/health	Reports application and database status.

RUNTIME PATH

Browser / ALB → Nginx :80 → Gunicorn 127.0.0.1:8000 → Flask app → RDS MySQL :3306

04 / COMPUTE & RUNTIME

Application Host Verification

The application-server terminal screenshot forms part of the host setup evidence. Application service and proxy checks were completed on the manually prepared application hosts before the corrected launch-template bootstrap was tested through Auto Scaling.

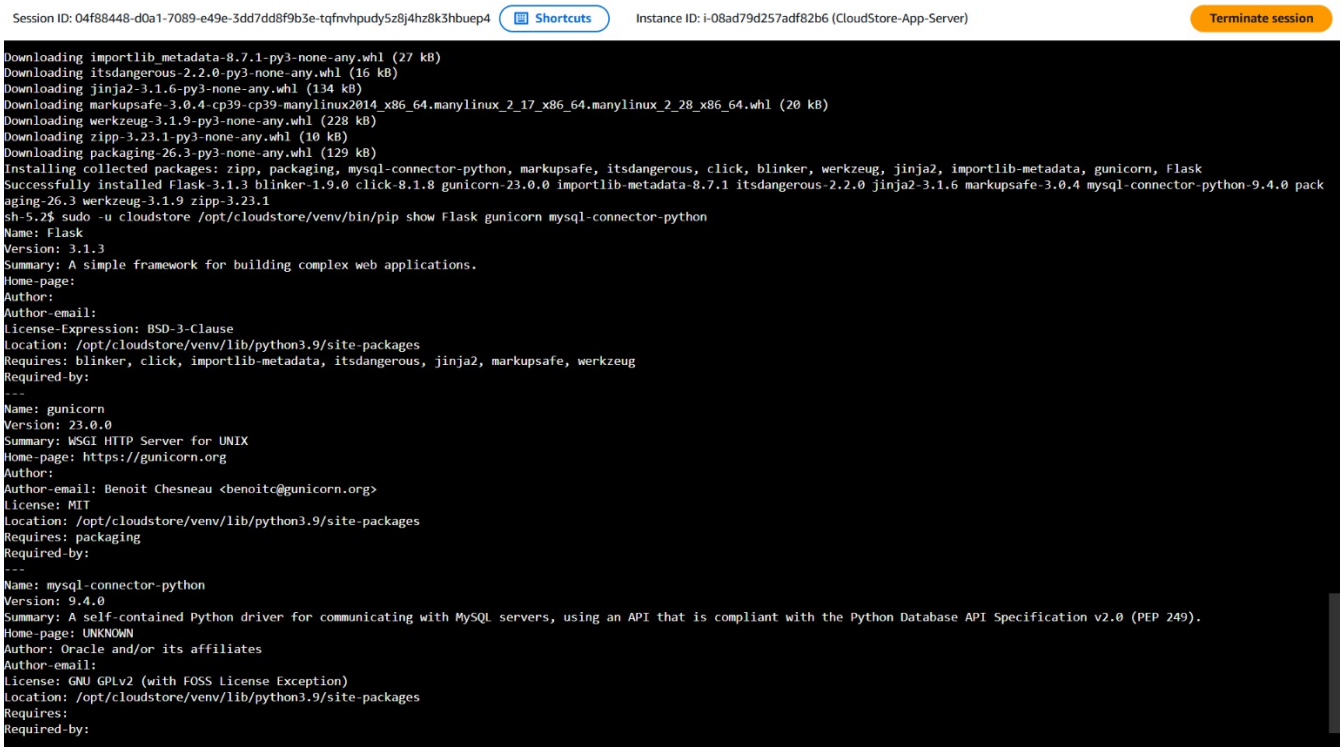


Figure. Application Host Verification - application host terminal evidence

The launch-template bootstrap initially failed because the JSON field names in user data did not match the keys stored in Secrets Manager. The template was updated to read DB_USER and DB_PASSWORD, and the default launch-template version was advanced to version 3.

Bootstrap dependency	Required configuration	Operational implication
Secrets Manager IAM access	secretsmanager:GetSecretValue	Instance role must retrieve the database secret.
Secret JSON keys	DB_USER and DB_PASSWORD	User data must parse the exact stored keys.
Systemd unit	cloudstore.service	Must start successfully for app checks to pass.
Nginx service	nginx.service	Must be enabled and listening on port 80.

Operational note: Auto Scaling instances should be treated as replaceable compute. All configuration required for a new instance must be reproducible from the launch template, repository, IAM instance profile and Secrets Manager secret.

05 / TRAFFIC MANAGEMENT

Load Balancer, Listener & Target Group

CloudStore-App-ALB is configured as an internet-facing Application Load Balancer across two Availability Zones. The captured listener view shows HTTP port 80 forwarding traffic to CloudStore-App-TG. The target group uses instance targets on HTTP port 80 and a /health health-check path.

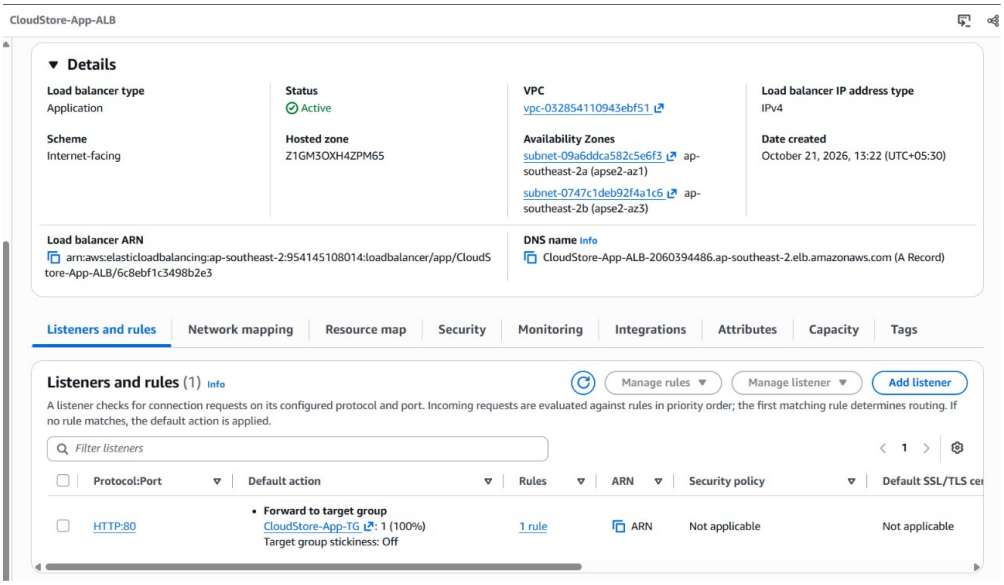


Figure 16. Application Load Balancer overview and listener forwarding to target group

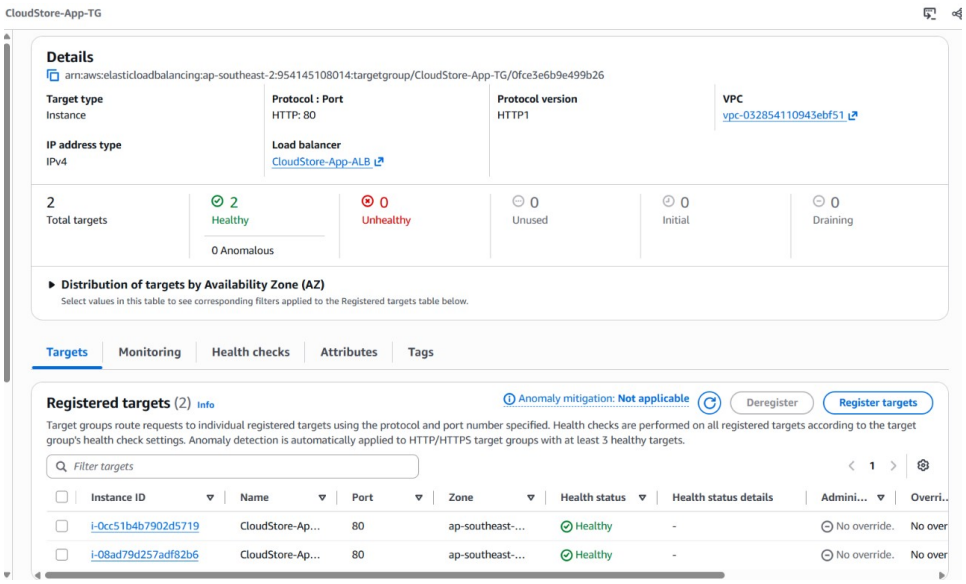


Figure 17. CloudStore-App-TG with two registered healthy targets

The supplied stable-state screenshot shows two healthy targets and zero unhealthy targets. The /health route returns the expected plain-text healthy response. The /api/health route separately reports application and database status.

Load-balancer element	Captured setting	Role
Scheme	Internet-facing	Accepts requests from external clients.
Listener	HTTP :80	Routes incoming web traffic.
Forward action	CloudStore-App-TG	Sends requests to registered EC2 targets.
Health check	HTTP /health, success code 200	Determines target availability.

06 / RESILIENCE & SECURITY

Auto Scaling, Monitoring & Security Boundaries

CloudStore-App-ASG uses a normal demonstration baseline of minimum capacity 2, desired capacity 2 and maximum capacity 4. The group spans two Availability Zones and is integrated with CloudStore-App-TG. The activity and instance-refresh evidence records replacement events and a successful rolling refresh.

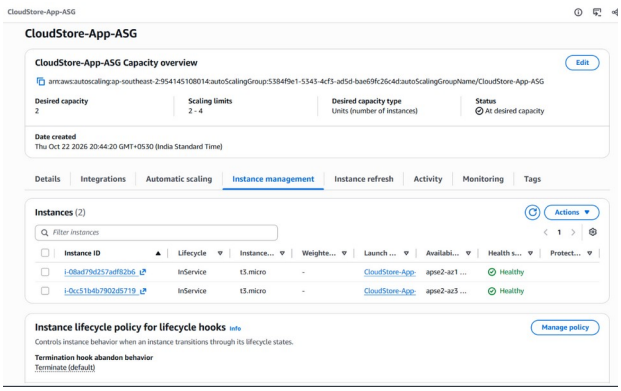


Figure 18. Auto Scaling Group capacity and two in-service instances

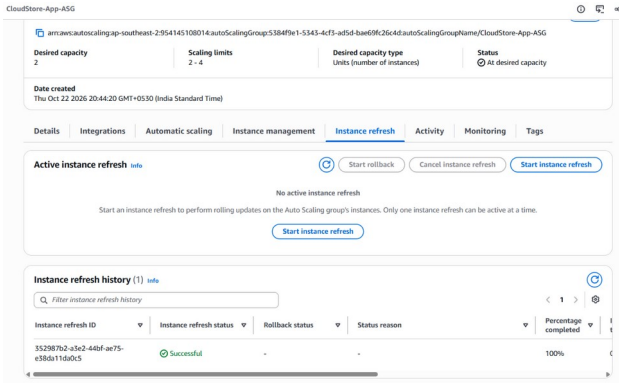


Figure 19. Instance refresh history showing successful completion

The tier security groups define this path: public HTTP/HTTPS ingress at the ALB; application HTTP ingress only from CloudStore-ALB-SG; and MySQL TCP 3306 ingress only from CloudStore-App-SG. This keeps the database outside the public ingress path.

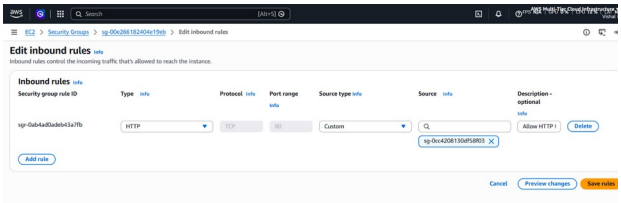


Figure 20. Application HTTP ingress scoped to the ALB security group

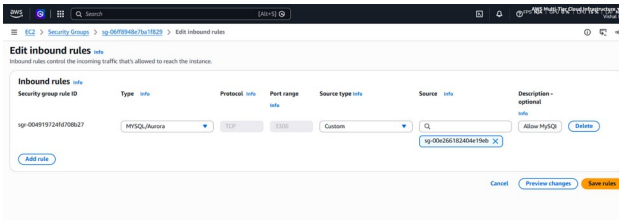


Figure 21. Database TCP 3306 scoped to the application security group

Tier	Security boundary	Observed rule
ALB	Public entry	HTTP 80 and HTTPS 443 from 0.0.0.0/0 in its security group.
Application	ALB-only ingress	TCP 80 from CloudStore-ALB-SG.
Database	App-tier-only ingress	TCP 3306 from CloudStore-App-SG.

HTTPS QUALIFICATION

The security group allows port 443, but the supplied ALB listener screenshot shows an HTTP:80 listener only. Do not describe HTTPS as active until an HTTPS listener and certificate are configured and verified.

07 / AWS CONSOLE EVIDENCE

Auto Scaling Capacity Test & Scaling Limits

The capacity-edit screenshot captures a controlled scale-out exercise. Desired capacity was temporarily changed from the normal two-instance baseline to three, while the minimum remained two and the maximum remained four. This stayed within the configured group limits and triggered Auto Scaling to create an additional application instance.

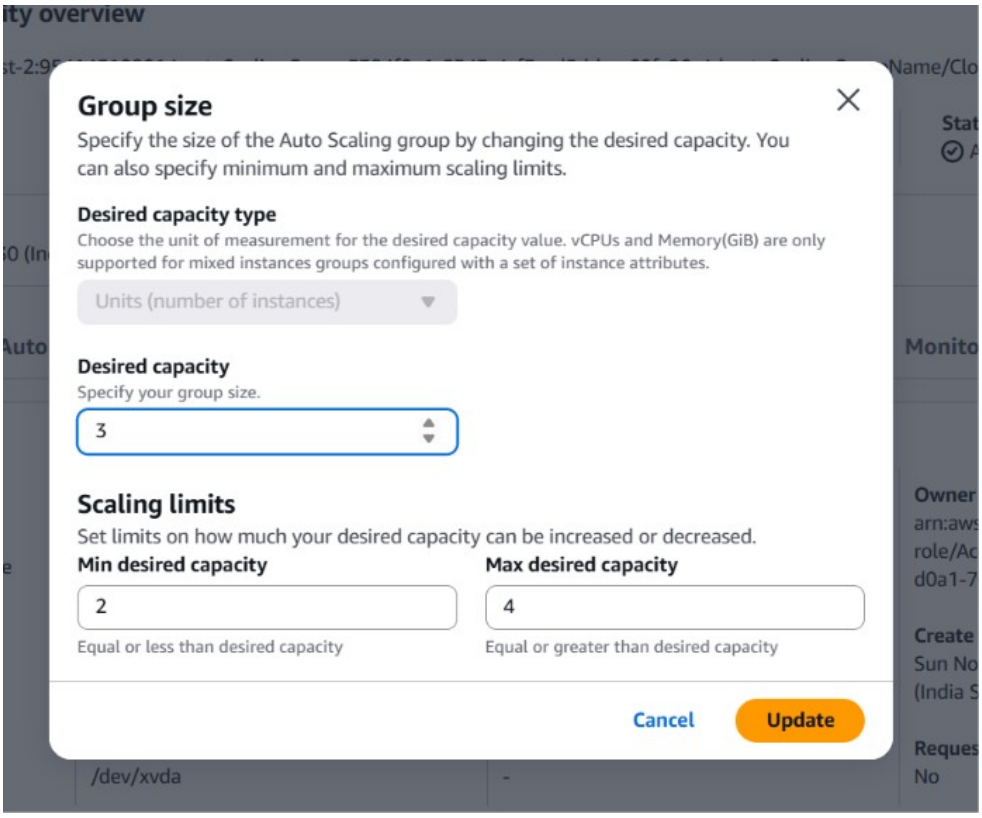


Figure 22. Auto Scaling group capacity dialog used for the temporary three-instance test

Capacity setting	Temporary test value	Normal demo baseline
Minimum	2	2
Desired	3	2
Maximum	4	4

The scale-out test demonstrates how desired capacity drives fleet size. The desired value is a target, not a promise that all instances are immediately ready; new targets pass through EC2 launch, system bootstrap, target registration and load-balancer health checks before receiving production traffic.

The expected scale-out sequence is: update desired capacity, wait for EC2 launch, allow user data to install and configure the application, register the instance with the target group, and wait for the target health check to pass. A launched EC2 instance is not considered application-ready until its target is healthy.

Lifecycle stage	What the operator checks	Ready condition
Launch	EC2 instance appears in the ASG instance list.	Instance reaches running state.
Bootstrap	User data log and systemd services.	CloudStore and Nginx are active.
Registration	Target group registered targets.	New target is registered on port 80.
Health	Target health status.	Target becomes Healthy.

COST CONTROL

After completing the test, the desired capacity was returned to 2. Keep the group at its required baseline when the project is not being demonstrated to avoid unnecessary EC2 charges.

07 / AWS CONSOLE EVIDENCE

Auto Scaling Group Network & Launch Template Binding

The ASG details view shows the group using the CloudStore application launch template and the two application subnets across ap-southeast-2a and ap-southeast-2b. The same view records the configured security group and the group network placement used by the application fleet.

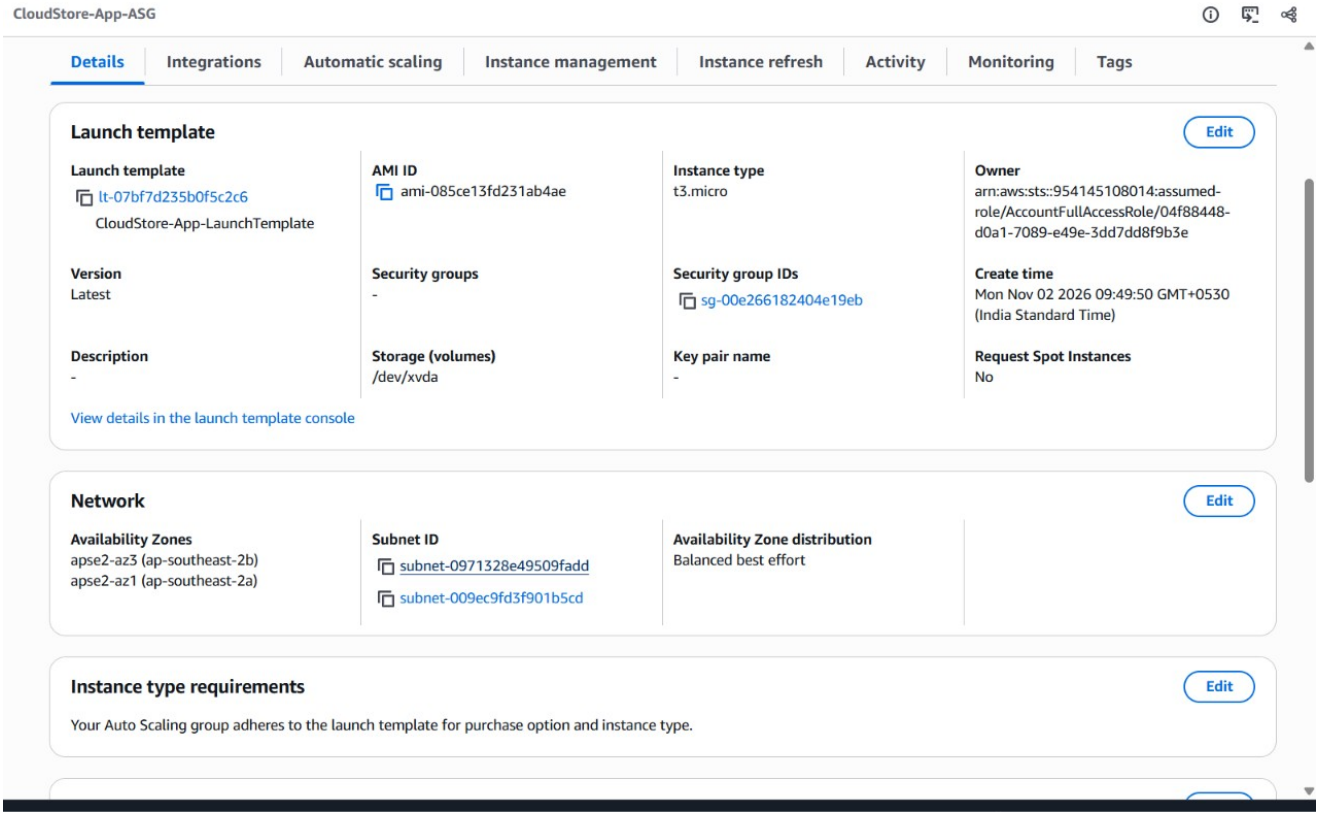


Figure 23. CloudStore-App-ASG details showing launch template and two Availability Zone subnets

Network / compute item	Observed configuration	Interpretation
Availability Zones	ap-southeast-2a and ap-southeast-2b	Application fleet is spread across two zones.
Subnets	Two application subnets	ASG can launch instances into either configured subnet.
Launch template	CloudStore-App-LaunchTemplate, latest version	New instances use the latest template version.
Instance type	t3.micro	Small general-purpose compute for the demo.

The security group is attached through the launch template. The app tier receives inbound HTTP from the ALB security group, while database access is defined separately on the RDS security group.

Network path	Source	Destination / port
Browser to load balancer	Public client	ALB listener on TCP 80.
Load balancer to app	CloudStore-ALB-SG	Application instances on TCP 80.
App to database	CloudStore-App-SG	RDS MySQL on TCP 3306.

This arrangement creates a clear ingress boundary for each tier. The web tier is the only public entry point; the app tier accepts traffic from the ALB security group; and the database tier accepts MySQL from the application security group only.

07 / AWS CONSOLE EVIDENCE

Target Group Health Check Configuration

The Target Group health-check settings use HTTP and the /health path on the registered target traffic port. A target must return the configured success code to be considered healthy. The screenshot shows a two-success healthy threshold, three-failure unhealthy threshold, five-second timeout and 30-second interval.

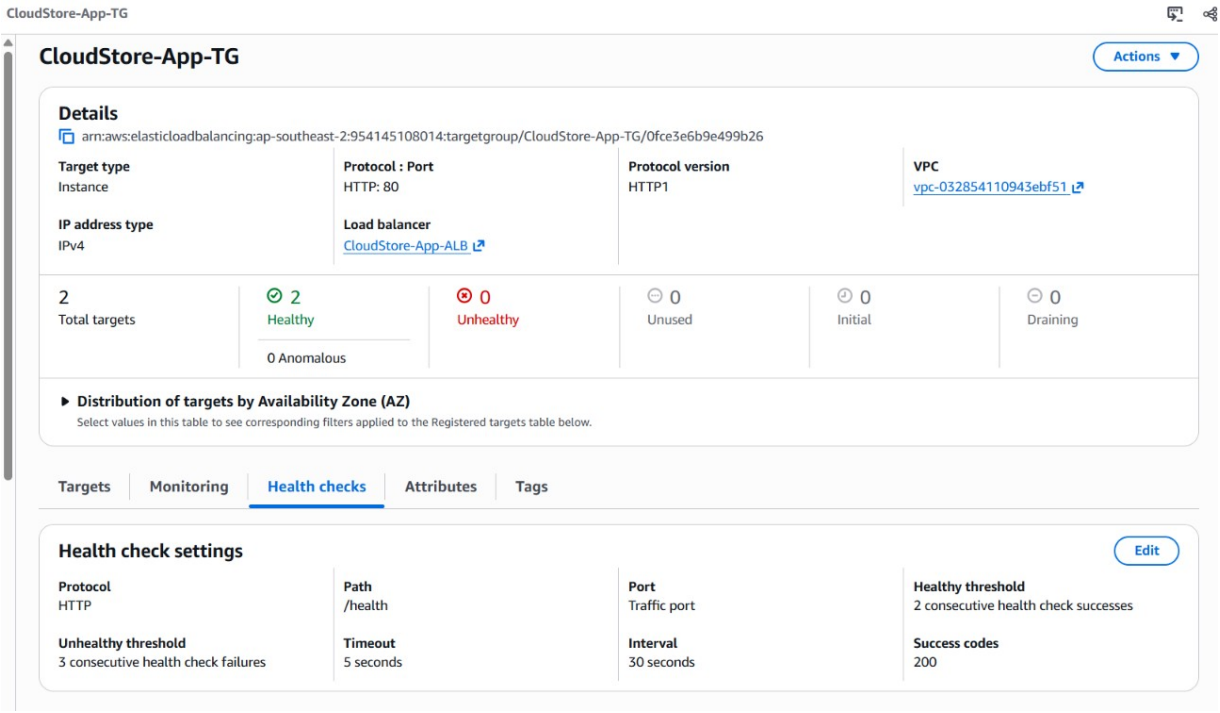


Figure 24. CloudStore-App-TG health-check protocol, path, threshold and timing settings

Health-check setting	Captured value	Operational meaning
Protocol / port	HTTP / traffic port	Probe follows the target group traffic port.
Path	/health	Tests a lightweight application endpoint.
Healthy threshold	2 consecutive successes	Requires repeated success before recovery.
Unhealthy threshold	3 consecutive failures	Marks a target unhealthy after repeated failures.
Timeout / interval	5 s / 30 s	Controls each probe and check frequency.
Success code	200	Only HTTP 200 is accepted in the captured configuration.

The health path should remain lightweight and should not depend on optional frontend assets. A failed check removes a target from normal routing while Auto Scaling can replace the unhealthy EC2 instance according to the group health policy.

Health state	What the ALB does	Operator follow-up
Healthy	Includes target in normal routing.	Confirm API and page responses through the ALB.
Unhealthy	Stops normal routing to the target.	Inspect target health details and instance bootstrap logs.
Initial	Waits for health-check evaluation.	Allow application startup and the configured success threshold.
Draining	Allows in-flight work to complete during deregistration.	Wait for target deregistration before cleanup.

08 / BOOTSTRAP AUTOMATION

Launch Template Versioning & Default Version Update

The Launch Template console first showed version 2 as the default and version 3 as the latest version. After the user-data correction, version 3 was selected as the default. This matters because Auto Scaling launches new instances from the template version configured by the group; the selected version must contain valid bootstrap logic.

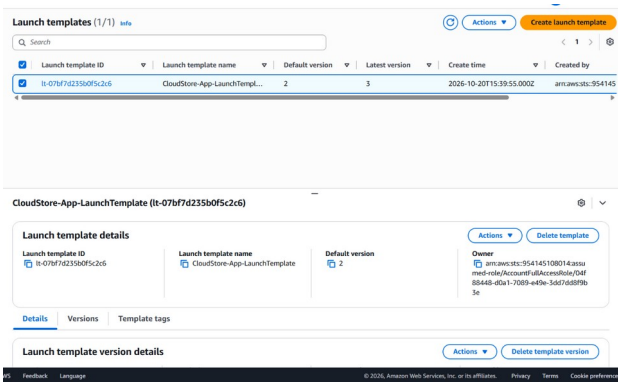


Figure 25. Launch Template version list before the default was switched to version 3

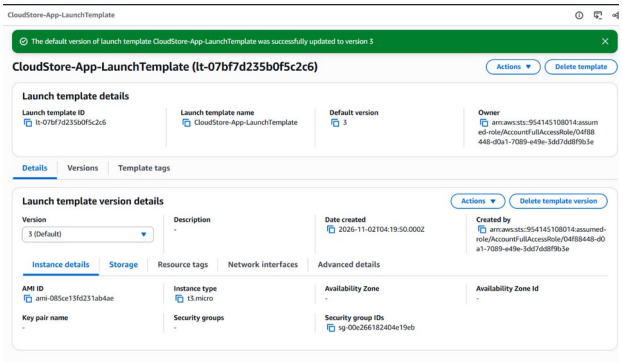


Figure 26. Console confirmation that version 3 became the default

Version	Observed state	Interpretation
1	Older version; initial configuration	Historical launch-template revision.
2	Default before correction	Previously selected default.
3	Latest and then default	Contains the corrected user-data secret-key parsing logic.

Versioning preserves a history of launch configuration changes and makes the correction auditable. The default version was updated successfully in the console, and the ASG details view subsequently showed that the group was using the latest launch-template version.

The corrected user-data script now reads DB_USER and DB_PASSWORD from the JSON secret stored in Secrets Manager. The earlier bootstrap run failed because the script asked for username and password keys that were not present in the secret. The report preserves that troubleshooting history because it explains why a new launch-template version was created.

Change-control stage	Action	Evidence checkpoint
Prepare	Update user-data to read the exact secret key names.	Script uses DB_USER and DB_PASSWORD.
Create version	Create a new launch-template version.	Version 3 appears in the Versions tab.
Promote	Set version 3 as the default.	Console success banner confirms the update.
Consume	ASG uses the latest launch-template version.	ASG Details view shows the current template reference.

CHANGE-CONTROL NOTE

When editing launch-template user data, create a new version, verify its contents, and only then select it as the default or update the ASG reference. Avoid editing a known-good version in place.

08 / BOOTSTRAP AUTOMATION

Secrets Manager & EC2 Instance Role

The EC2 instance role is CloudStore-EC2-Instance-Role. The IAM role screenshot lists the managed policies for Systems Manager and CloudWatch and a customer inline policy named CloudStore-SecretsManager-ReadOnly. That inline permission is intended to allow the instance to retrieve the one CloudStore database secret required by the bootstrap script.

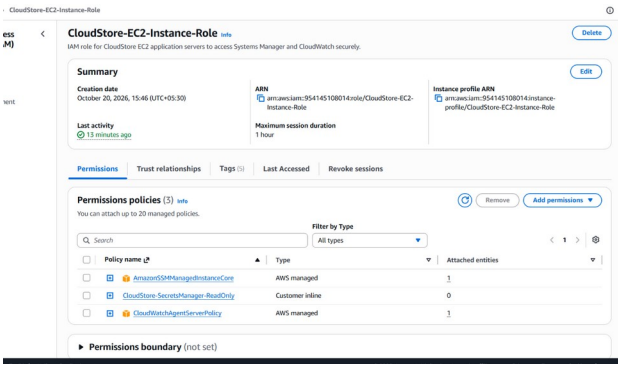


Figure 27. CloudStore EC2 role and attached permission policies

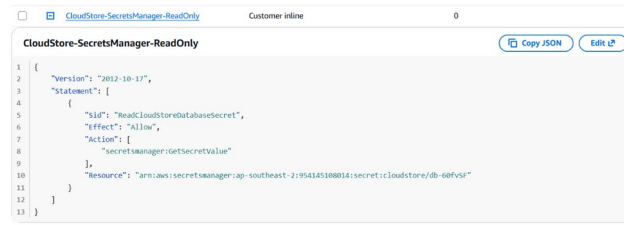


Figure 28. Inline policy scoped to Secrets Manager GetSecretValue for the CloudStore DB secret

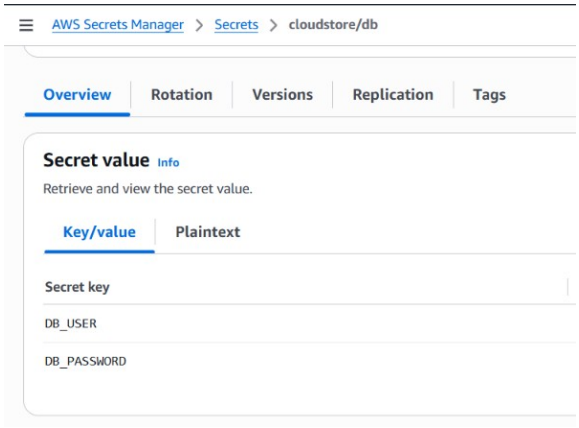


Figure 29. Secrets Manager secret key names shown without exposing secret values

Secret / permission item	Observed detail	Why it is used
Secret identifier	cloudstore/db (secret ARN scoped in policy)	Names the database secret resource.
Secret JSON keys	DB_USER and DB_PASSWORD	Matches the keys expected by the corrected user data.
IAM action	secretsmanager:GetSecretValue	Allows retrieval of the secret string.
Access boundary	Specific secret resource	Avoids broad access to unrelated secrets.

The intended startup sequence is: the EC2 instance receives its IAM instance profile, the bootstrap script requests the named secret, the instance role authorises the GetSecretValue action for the scoped resource, and the script writes a protected environment file for the CloudStore service. This avoids placing the database password directly in the launch template.

Control	Expected behaviour	Security benefit
Instance profile	Attach CloudStore-EC2-Instance-Role to launched EC2 instances.	Provides short-lived AWS credentials to the host.
IAM action	Permit GetSecretValue for the one CloudStore secret.	Limits secret retrieval scope.
Secret fields	Use DB_USER and DB_PASSWORD.	Keeps code aligned with the stored JSON schema.
Local file	Restrict env-file access to root and cloudstore group.	Protects runtime credentials on the instance.

09 / INSTANCE LIFECYCLE

Instance Refresh Strategy & Rollout Settings

The Instance Refresh console view records a rolling refresh started for CloudStore-App-ASG. The captured settings show a minimum healthy percentage of 50%, an instance warm-up of 300 seconds, skip matching enabled and CloudWatch alarm integration enabled. The screenshot itself shows the refresh in a pending state at the time it was captured.

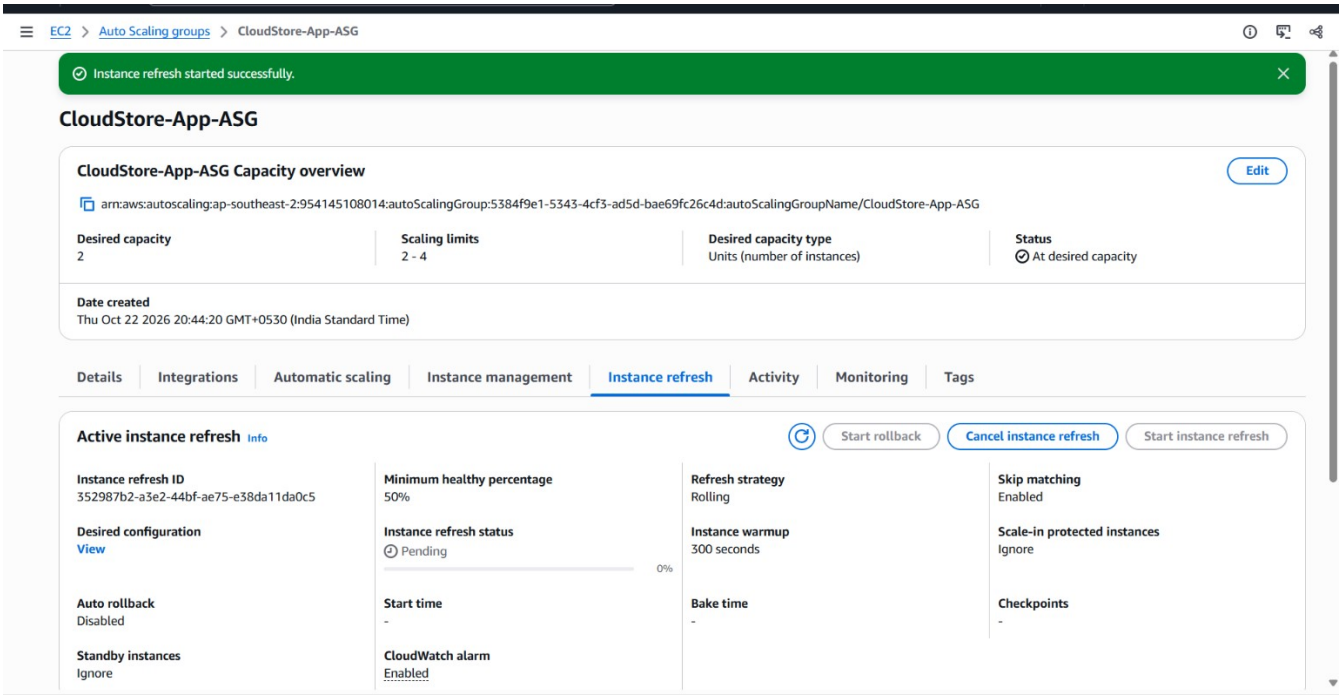


Figure 30. Instance Refresh pending state with rolling strategy, 50% minimum healthy and 300-second warm-up

Setting	Captured value	Meaning
Refresh strategy	Rolling	Instances are replaced progressively.
Minimum healthy	50%	Allows at least half the desired fleet to remain healthy during replacement.
Instance warm-up	300 seconds	Allows a new instance time to initialise before metrics/refresh progression.
Skip matching	Enabled	Avoids replacing instances already matching the desired configuration.
CloudWatch alarm	Enabled	Allows alarm monitoring to participate in refresh evaluation.

The pending screen should not be treated as proof of completion. The following pages use the later instance-refresh history and activity records, which show successful completion and individual replacement events.

Before starting a refresh, the operator should verify that the new launch-template version is correct, the target group is healthy, and the application can reach the RDS database. If a replacement instance fails bootstrap, the refresh may stall or unhealthy targets may be replaced repeatedly.

Pre-refresh check	Why it matters	Evidence used in this report
Launch-template version	Prevents repeating a known bootstrap failure.	Version 3 selected as default.
Target group baseline	Provides a known healthy starting point.	Two healthy targets were captured before the refresh.
Database secret permission	Allows new instances to complete app configuration.	IAM inline policy grants scoped secret retrieval.
Application health	Confirms app and DB checks are available.	/api/health returned application=ok and database=ok.

09 / INSTANCE LIFECYCLE

Successful Instance Refresh & Replacement Activity

The later refresh-history screenshot shows the refresh in a Successful state at 100% completion. The Activity history records successful launch and termination events during the refresh, including launch of a new EC2 instance and retirement of an older instance after replacement.

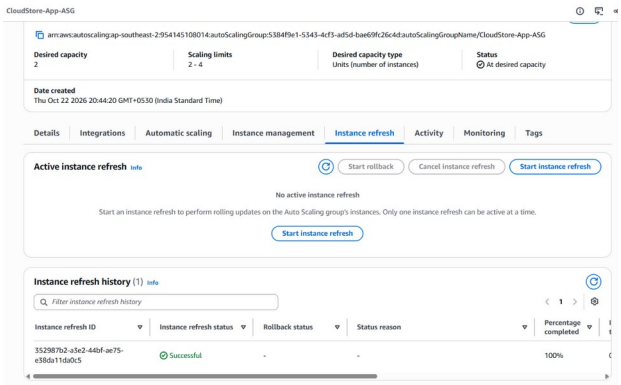


Figure 31. Instance refresh history showing Successful status at 100%

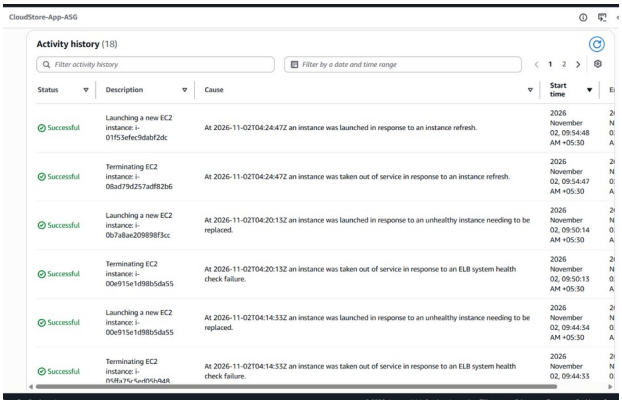


Figure 32. Auto Scaling Activity history showing refresh launches and terminations

Observed event	Console evidence	Interpretation
Launch new EC2 instance	Successful activity entry	ASG provisioned replacement capacity.
Terminate old EC2 instance	Successful activity entry	Previous instance was removed after replacement.
Refresh history	Successful; 100%	The rolling refresh completed.
Target health after refresh	Two registered targets shown healthy in the later target-group capture	New application fleet returned to a healthy routing state.

A rolling refresh reduces the need for a simultaneous fleet replacement. The warm-up period gives each new instance time to complete bootstrap and register healthy before the group proceeds to the next replacement.

Operator verification	What successful behaviour looks like	Why it is checked
Capacity	The ASG returns to desired capacity after replacement.	Ensures the application fleet is restored.
Target health	New targets become Healthy.	Prevents routing to a failed or incomplete app server.
Refresh status	History reports Successful at 100%.	Provides a clear completion record.
Activity history	Launch and terminate events report Successful.	Shows the lifecycle steps completed.

The activity history also contains earlier launch and termination attempts associated with unhealthy replacements. Those earlier events explain why the launch-template bootstrap and health-check sequence needed troubleshooting; they should not be confused with the final refresh result. The captured refresh-history entry is the final completion evidence.

Historical signal	How to read it	Operator response
Earlier unhealthy replacement	An instance did not become a usable target.	Inspect cloud-init, service status and target health details.
Corrected launch-template version	Version 3 became the default after secret-key parsing was corrected.	Use the corrected default for subsequent launches.
Final refresh result	Successful status at 100%.	Treat the captured rolling refresh as completed.
Stable target group	Two targets were later shown Healthy.	Confirm the application fleet recovered after refresh.

10 / MONITORING

CloudWatch Auto Scaling Metrics & Capacity Signals

Auto Scaling group metrics collection is enabled in the captured Monitoring tab. The first dashboard image includes minimum and maximum group size, desired capacity, in-service instances and pending/terminating instance counts. The second image shows capacity-unit and warm-pool metrics. Warm-pool values remain at zero in the captured period because no warm pool was configured.

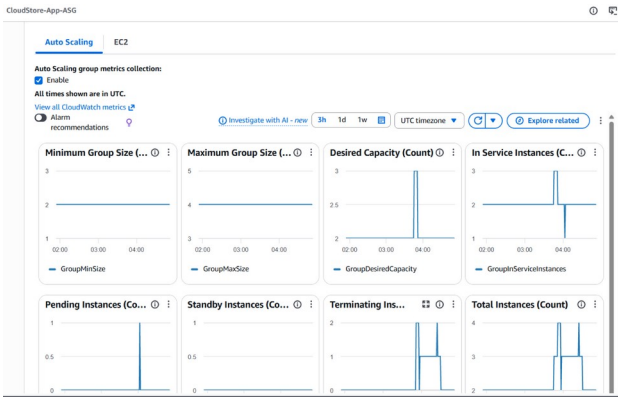


Figure 33. ASG CloudWatch graphs for desired capacity and instance lifecycle counts

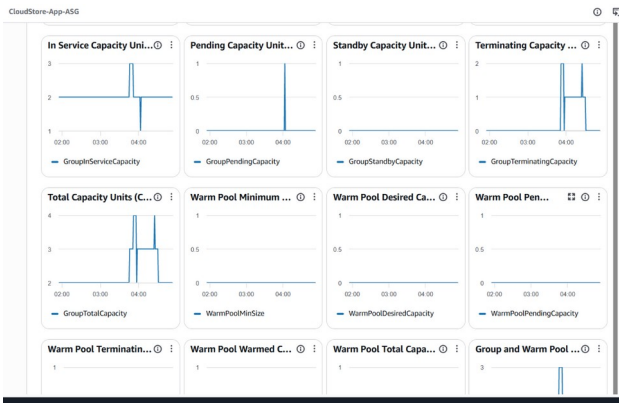


Figure 34. Additional capacity-unit and warm-pool metrics

Metric family	Captured purpose	How to interpret
GroupMinSize / GroupMaxSize	Displays configured capacity boundaries.	Compare desired capacity with permitted limits.
GroupDesiredCapacity	Shows the target instance count over time.	A short increase to 3 corresponds to the scale-out test.
GroupInServiceInstances	Shows the count of serving instances.	May lag desired capacity while instances launch or terminate.
Pending / Terminating	Shows lifecycle transitions.	Non-zero values are expected during replacement events.
Warm-pool metrics	Shows warm-pool capacity.	Zero is consistent with no warm pool configured.

The graphs are evidence of control-plane telemetry, not a full application performance test. CPU, request latency, error rate and database load would need their own metrics and alarms for a broader operational monitoring design.

Monitoring question	Metric or source	What it can show
How many instances should run?	GroupDesiredCapacity	The target size requested from Auto Scaling.
How many are serving?	GroupInServiceInstances	The number currently in the in-service lifecycle state.
Is replacement underway?	Pending / Terminating counts	Whether instances are being launched or removed.
Is a warm pool in use?	WarmPool capacity metrics	A zero line is consistent with no warm pool configured.

For a more complete operations view, Auto Scaling metrics should be paired with application-level signals. Group counts describe fleet state, but they do not tell an operator whether customers are receiving slow responses or encountering application exceptions. The next layer of monitoring should connect service-level symptoms to the underlying compute and database health.

Recommended alarm area	Metric / source to add	Operational response
Compute saturation	EC2 CPUUtilization and status checks.	Review scaling policy and instance health.
Application availability	ALB HTTPCode_Target_5XX_Count and HealthyHostCount.	Inspect target logs, health endpoint and deployment status.
Request performance	ALB TargetResponseTime.	Check Nginx, Gunicorn worker saturation and database response.
Database pressure	RDS CPUUtilization, DatabaseConnections and FreeStorageSpace.	Review query load, connection use and storage headroom.

11 / NETWORK SECURITY

Security Group Boundaries: ALB, App and Database

The supplied security-group console screenshots demonstrate a source-restricted tier design. CloudStore-ALB-SG is the public-facing web entry group. CloudStore-App-SG allows HTTP port 80 only from the ALB security group. CloudStore-DB-SG allows MySQL port 3306 only from the application security group.

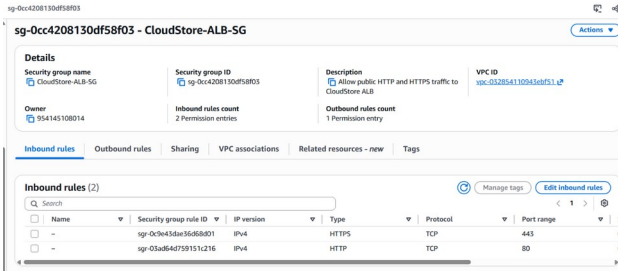


Figure 35. ALB security group summary

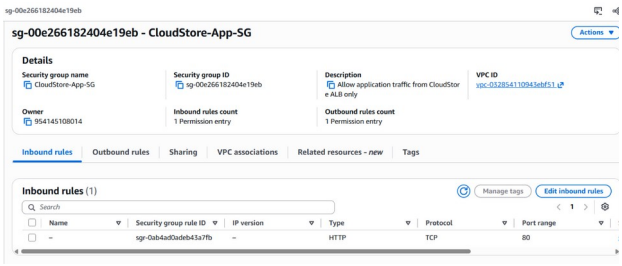


Figure 36. Application security group summary

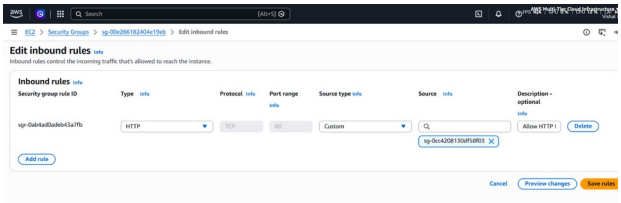


Figure 37. Application HTTP ingress source set to the ALB security group

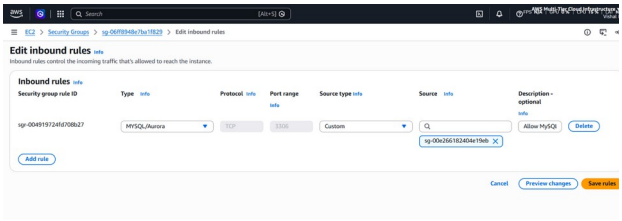


Figure 38. Database MySQL ingress source set to the application security group

Security group	Inbound rule	Source / boundary
CloudStore-ALB-SG	HTTP 80; HTTPS 443	0.0.0.0/0 public web ingress.
CloudStore-App-SG	HTTP 80	CloudStore-ALB-SG only.
CloudStore-DB-SG	MySQL/Aurora TCP 3306	CloudStore-App-SG only.

Security groups are stateful virtual firewalls. The rule source should reference the upstream security group for private tier-to-tier communication rather than exposing internal services to the full internet. In this design, the app servers do not need a public inbound rule for port 80, and the RDS database does not need a public inbound rule for port 3306.

Review item	Expected safe design	Captured result
Public access	Only the ALB receives public web traffic.	ALB-SG allows public web ingress.
Application ingress	Only the ALB SG can reach application port 80.	App-SG source references ALB-SG.
Database ingress	Only the application SG can reach MySQL 3306.	DB-SG source references App-SG.
TLS listener	ALB listener on 443 with a valid certificate.	Not shown in supplied listener screenshot.

HTTPS QUALIFICATION

The ALB security group includes port 443, but the listener evidence supplied for this report shows HTTP:80 only. Add and validate an HTTPS listener with a trusted certificate before presenting HTTPS as active.

12 / FINAL VERIFICATION

End-to-End Validation & Project Handover

The final validation combined browser testing, direct API requests through the ALB DNS name, target-group health status and Auto Scaling lifecycle evidence. The supplied PowerShell output shows HTTP 200 for /health, /api/health, /api/categories, /api/products and /api/products/1. Ten consecutive /api/health requests also returned HTTP 200.

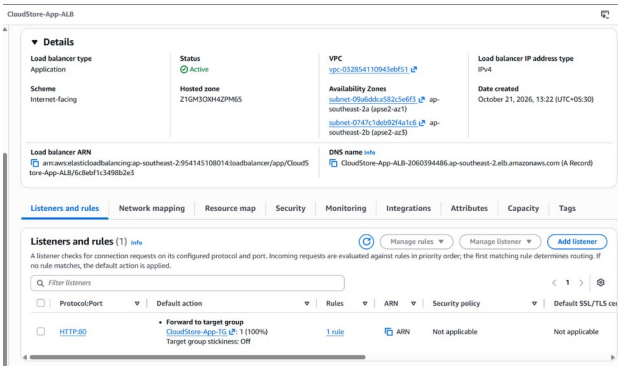


Figure 39. Internet-facing ALB listener routes traffic to CloudStore-App-TG

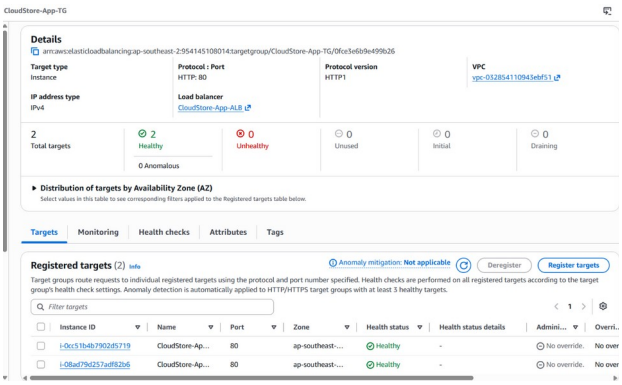


Figure 40. Target group showing two healthy application targets

Validation area	Observed result	Status from supplied evidence
Homepage	HTTP 200; storefront rendered in browser	Passed
Static CSS / JavaScript	HTTP 200	Passed
Health endpoint	HTTP 200; healthy	Passed
Application + DB health	HTTP 200; application=ok, database=ok	Passed
Categories / Products	HTTP 200; six categories and 18 products	Passed
Product detail	HTTP 200 for /api/products/1	Passed
Repeated health requests	10 of 10 returned HTTP 200	Passed
Cart and browse interactions	User confirmed category/search/product/cart controls worked	Passed
Demo order submit	Temporary client-side reset error; no order completion confirmed	Not passed / excluded

The project is ready to be presented as an AWS multi-tier CloudStore portfolio implementation with a deployed storefront, application and database tiers, load balancing, Auto Scaling, instance refresh, Secrets Manager and restricted security-group rules. State the checkout limitation and the absence of a verified HTTPS listener when discussing the current demonstration status.

Next improvement	Reason	Acceptance check
Configure HTTPS	Public listener evidence currently shows HTTP:80 only.	HTTPS listener uses a trusted certificate and redirects or routes correctly.
Fix demo checkout reset issue	Order submission showed a browser-side reset error.	A test order returns an API success response and is visible in MySQL.
Add application-level alarms	ASG metrics are captured, but app latency/error alarms are not demonstrated.	CloudWatch alarms cover 5xx, latency and instance health.
Review secret rotation	Credentials should not remain exposed if previously pasted into logs.	Rotate the DB app password and update Secrets Manager securely.

HANDOVER CHECKLIST

Keep Auto Scaling desired capacity at 2 when idle; rotate any database credential previously exposed in chat or logs; avoid committing secrets; retain the latest launch-template default version; and re-check target-group health after future configuration changes.